

The CUDA ClusterFinder

Kernel design · hardware limits · seven optimization steps

Khalil Daniel Ferjaoui

The kernel was never the bottleneck —^{*} feeding it was

One kernel, one thread per pixel, and seven steps to keep it fed

The stencil was fast almost immediately: at 3×3 the kernel needs 5.5 μs per frame while getting that frame across PCIe costs 16.6 μs , or 13.2 uncontended. The frame is gated by PCIe, not the arithmetic. Six of the seven steps get data in, get results back, and measure honestly.

WHERE THIS ENDS UP · THE SHIPPED f32 BUILD AFTER ALL SEVEN STEPS · ENGINE TIMES [f32 · s4] · RECONCILED IN A1

×9.1

VS 24-THREAD CPU

61,312

FRAMES / SECOND

16.3 μs

PER FRAME, END TO END

6 / 23 M

CLUSTER MISMATCH VS CPU
TWIN

RTX 4090 (Ada, sm_89) · PCIe 4.0 ×16 · Mönch 400×400 uint16 · 3×3 clusters · 100 000 frames · Cu fluorescence, MAX IV

* This em dash was hand written (not AI generated ☺)

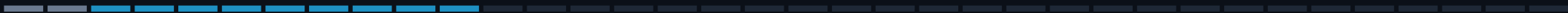
CONTEXT · WHAT THE CODE DOES

What the kernel does, and what limits it

No optimizations yet, only what the hardware has to do.

COMING UP

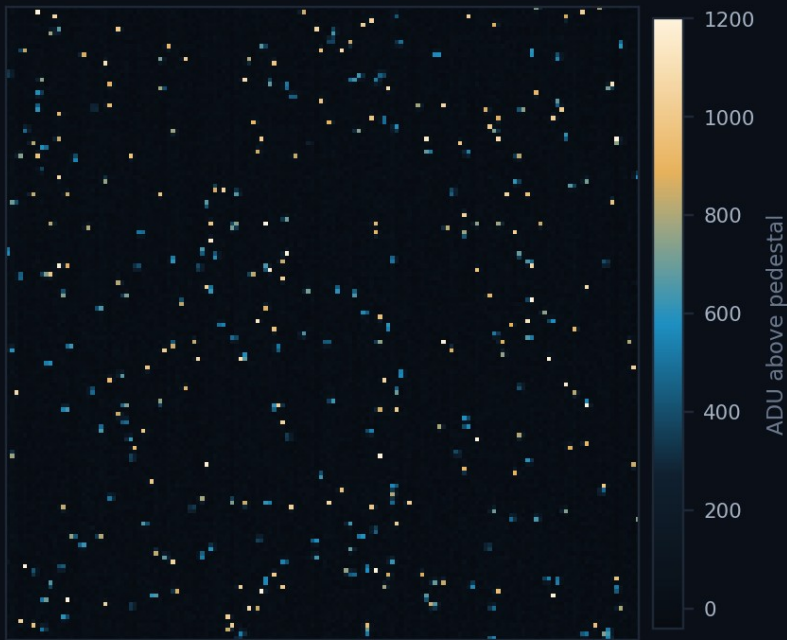
- 3–4 The algorithm
- 5–6 The two machines
- 7–8 The CUDA kernel
- 9–10 What limits it
- 11 The roadmap



A photon is not a pixel — it is a cluster

- Charge from one absorbed photon **spreads over neighbouring pixels**. Summing that 3×3 patch recovers the photon energy; a single pixel does not.
- The histogram of those cluster energies **is** the measurement: peak position and width give the detector's gain and **energy resolution**.

one frame, pedestal subtracted · 150×150 crop



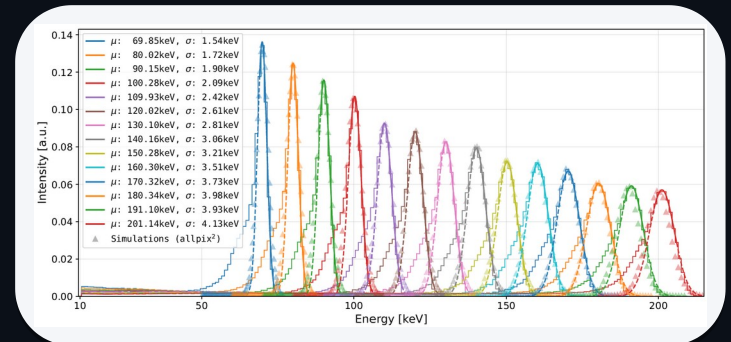
9×9 zoom on one hit



3×3 sum = 2258 ADU — one photon.
The peak pixel holds only part of the charge.

Real MOENCH data, Cu fluorescence, MAX IV beamtime. One cluster is emitted per local maximum, so 2 330 counts photons, not lit pixels; the recorded 3×3 windows cover 12.7 % of the frame. The detector's frame rate is the number every throughput figure in this deck should be read against.

WHY IT MATTERS



Cluster-energy spectra from an energy scan, against allpix² simulation. Each peak is one beam energy; its width is the resolution being measured.

MÖNCH03 · THE DETECTOR THIS FEEDS

ARRAY · PITCH · ACTIVE AREA

400 × 400 · 25 μm · 10 × 10 mm²

FRAMES PER SECOND

1.3 k standard, 3-6 k optimised

PEAK PIXELS = PHOTONS / FRAME

~2 330 · 1.5 %

Per pixel: subtract, threshold, update the pedestal

- Per pixel: subtract a **running pedestal** (mean $\pm$ rms), keep pixels above **$n\sigma \cdot \text{rms}$** , cut a 3×3 cluster around each local maximum.
- 400×400 = 160 k pixels, **312.5 kB per frame**; Cu data yields ~2 330 clusters per frame at 3×3.
- The pedestal is **updated by every pixel that sees no photon**, about 80 % of them, every frame, so the arithmetic and the data movement are coupled.

THE WHOLE ALGORITHM · THREE OUTCOMES, NOT TWO

```
// the whole algorithm, per pixel
v  = frame[i] - pedestal_mean[i]
rms = pedestal rms at i
m  = max(v) over the 3x3 window at i
if (m > nSigma * rms)           // a photon is within reach
    if (v == m) -> emit cluster    // ... and I am its peak
    else       -> nothing         // ... I am in its shadow
else                          -> update pedestal    // I saw nothing
```

Thesis of this talk: the compute was fast almost immediately. Six of the seven steps are about feeding it.

THE SHAPE OF THE WORK

WORK ITEMS PER FRAME

160 000

OPERATIONS ON EACH

~5, identical

COMMUNICATION BETWEEN THEM

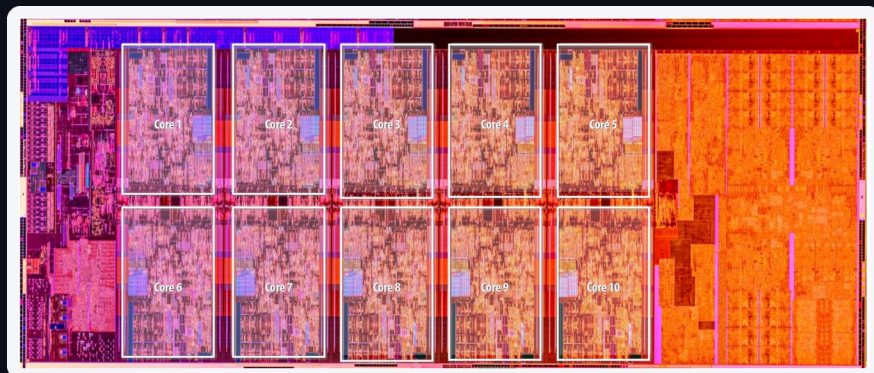
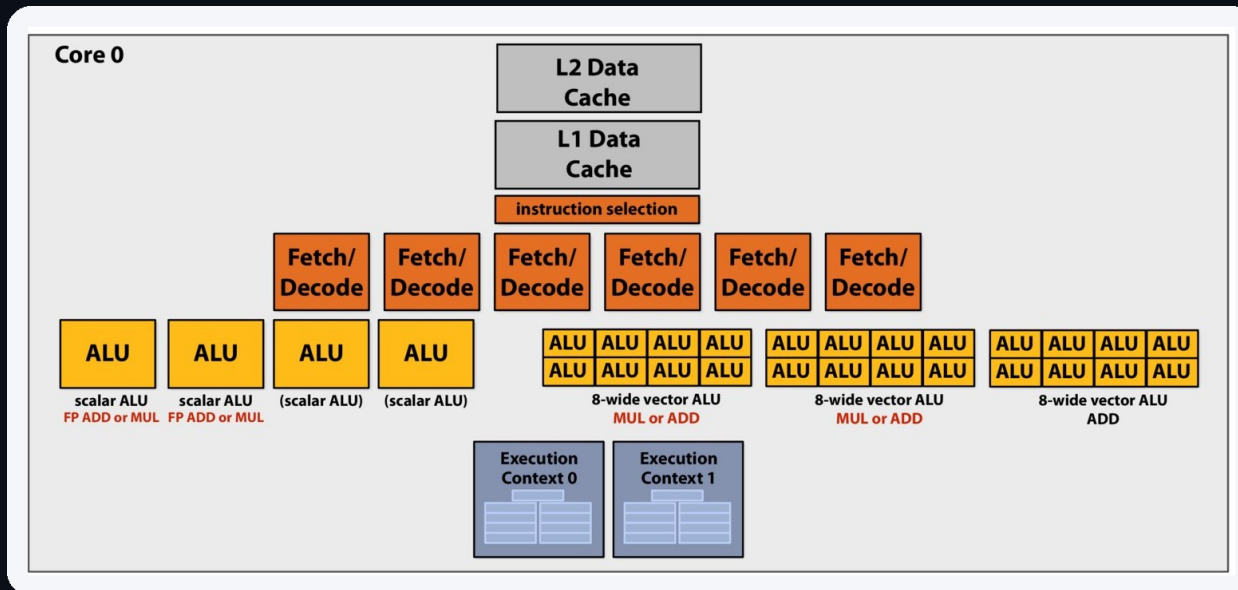
none

ORDER THEY MAY RUN IN

any, once the pedestal is fixed

A pixel reads what its neighbours MEASURED, never what they decided. With the pedestal held at its frame-start value, order cannot matter. That is what licenses one thread per pixel. The serial CPU updates the pedestal mid-scan and so gives that property up; annex A7 measures what it costs.

CPU: latency-oriented, built to finish one thread fast



Count the boxes: **6 fetch/decode**, out-of-order instruction selection, two levels of private cache, all of it to keep **two** instruction streams fed. The ALUs are the small part.

Comet Lake · 10 cores boxed; nearly half the die is cache and I/O.

Both diagrams after Stanford CS149, Fall 2025.

PC-MOENCH-04 · AMD RYZEN 9 7950X

CORES × SMT

16 × 2

CPU, 1 THREAD

1.75 ms

CPU MT, 24 THREADS

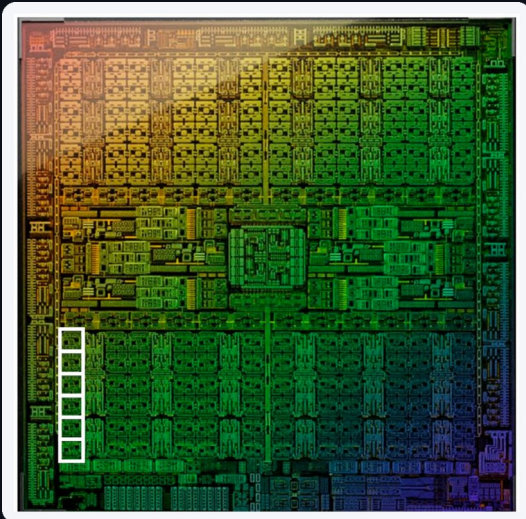
148 μs

THAT IS THE BAR

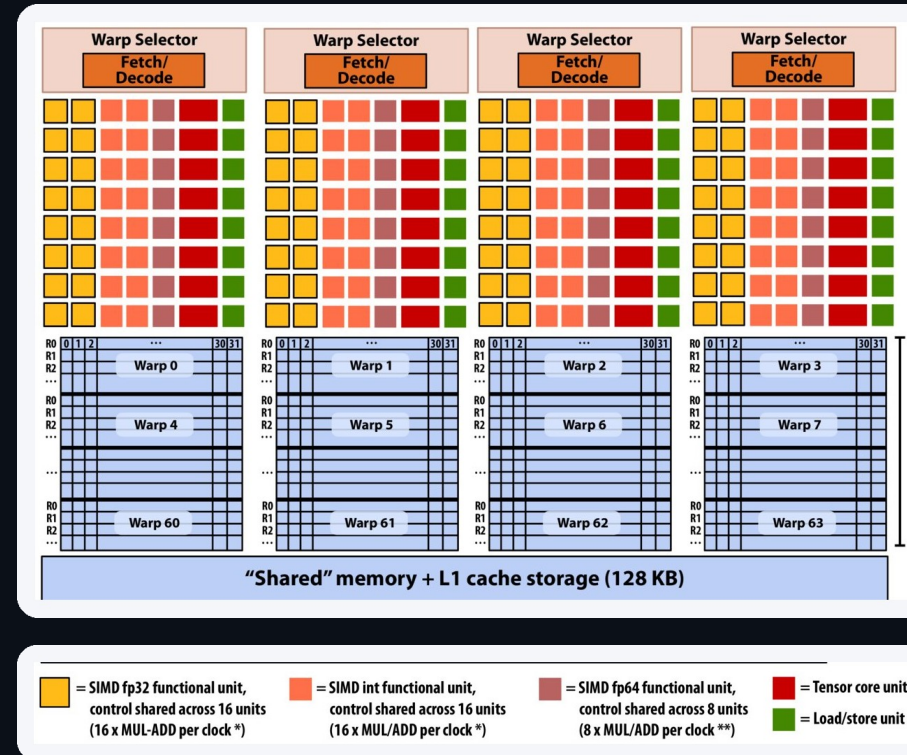
6 762 frames / s

Thread count is swept per cluster size, not assumed: 24 at 3×3, 32 at 9×9.

GPU: throughput-oriented, the whole frame at once



AD102 · 144 blocks, 128 enabled on this card. One SM boxed.



NVIDIA GEFORCE RTX 4090

FP32 LANES

16 384

128 SMS × 128 LANES

vs 32 CPU streams

RESIDENT THREAD SLOTS

196 608

THE FRAME NEEDS

160 000 → all at once

Device memory runs at 1 008 GB/s (384-bit, 21 Gbps); PCIe delivers 25. Two copy engines, so H2D and D2H run at the same time.

Same grammar, inverted proportions: **4 fetch/decode** for **64 warp contexts** and a wall of lanes.
Nothing reorders instructions: when a warp stalls on memory, the selector just **runs a different one**.

One SM: a V100 is shown; this card's is the same idea: 128 FP32 lanes, 48 warp slots, 100 kB shared memory. After Stanford CS149, Fall 2025.

One thread per pixel

1 load tile + halo

>

2 __syncthreads

>

3 stencil reduction

>

4 classify

>

5 append or update pedestal

- A **16×16 block = 256 threads** covers 256 pixels; the grid tiles the whole 400×400 frame, 625 blocks of identical work.
- The output is **sparse**: only detections touch global memory, through one atomic bump of a per-frame counter. The **decision work is dense**: every pixel is tested, independently and identically.
- That is exactly the shape a GPU wants: regular, independent, repeated 160 000 times per frame.

LAUNCH CONFIGURATION · ClusterFinderCUDA.hpp

```
block = dim3(BLOCK_X, BLOCK_Y);           // 16 x 16
grid  = dim3((ncols + BLOCK_X - 1)/BLOCK_X,
             (nrows + BLOCK_Y - 1)/BLOCK_Y);
device::find_clusters_in_single_frame<<<grid, block, shmem, stream>>>(
    d_frame, d_pd_mean, d_pd_sum, d_pd_sum2, d_pd_off, n_pd_samples,
    nSigma, nrows, ncols, d_clusters, d_cluster_count, max_clusters);
```

400×400 pixels → a 25×25 grid of 16×16 blocks = 625 blocks per frame, handed to 128 SMs. Nothing about the launch depends on how many clusters the frame happens to contain, which is what makes the work uniform.

THREE OUTCOMES, NOT TWO

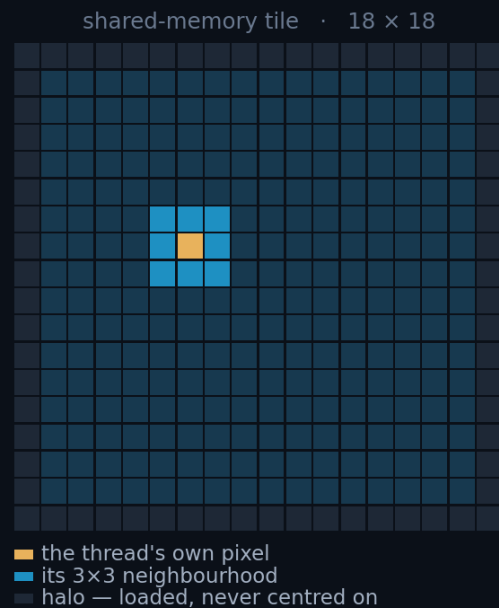
```
// ClusterFinder.hpp · the serial CPU
if (max > nSigma * rms) {
    if (value < max)
        continue;
    // Not max go to the next pixel
    // but also no pedestal update
} else {
    pedestal.push_fast(iy, ix, ...);
}
```

Verbatim from the CPU source. A shadow pixel is **neither recorded nor fed back**.

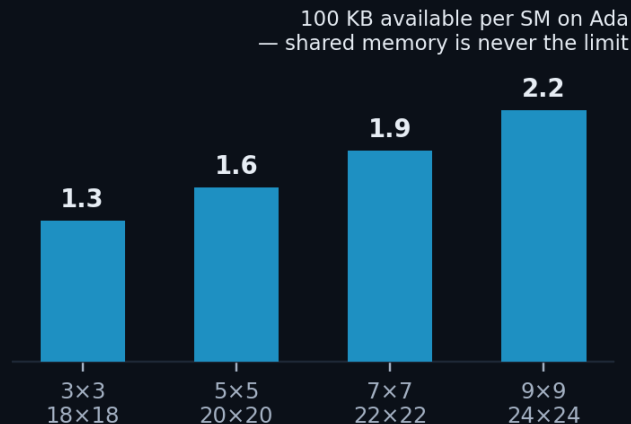
~80 % of threads update the pedestal, ~18 % are shadow, ~1.5 % peaks. The update dominates, not the write.

Load the tile once, reuse it nine times

- Neighbouring threads need **overlapping** 3×3 windows. Without shared memory each pixel would be fetched from global memory up to nine times.
- Each block stages a tile of $(16 + 2r) \times (16 + 2r)$ **pedestal-subtracted** values, the halo is the price of the stencil, and it is loaded cooperatively by the threads on the block edges.



KB of shared memory per 16×16 block (float tile)



clusterfinder_kernel.cuh

```
extern __shared__ unsigned char smem[];
auto *sh = (COMPUTE_TYPE*)smem;
auto stride = blockDim.x + 2*col_radius;
auto tid = (threadIdx.y + row_radius)*stride
          + (threadIdx.x + col_radius);
// pedestal subtraction fused into the load
sh[tid] = d_frame[gid] - d_pd_mean[gid];
```

Only **odd** cluster sizes are supported (3×3 , 5×5 , 7×7 , 9×9), so that the centre pixel is unique and local-maximum suppression is well defined.

The tile is stored in COMPUTE_TYPE (float), not in the pedestal type: 1.3 KB for 3×3 , 2.3 KB for 9×9 , against 100 KB of shared memory per SM. Even the old double-precision tile only reached 4.5 KB.

Halo cost falls with block size: 56 % of the tile at 8×8 , 27 % at 16×16 , 13 % at 32×32 , which is the first half of the block-size argument. The second half is registers, next slide.

38 registers per thread at 3×3, 128 at 9×9

- An SM has a fixed budget: **65 536 registers** and **1 536 thread slots**. A 16×16 block claims 256 slots and $256 \times (\text{registers per thread})$, and whichever budget runs out first decides how many blocks fit on the SM at once.
- Every thread keeps a private **clusterData[CSX × CSY]** staging array, so register demand grows with the **square** of the cluster size. Neither build spills: ptxas would rather hold fewer blocks than go to local memory.



At 9×9 the register file is exactly full at two blocks ($2 \times 128 \times 256 = 65\,536$), so two thirds of the thread slots are stranded. At 3×3 the slots run out first and the registers still have room.

MEASURED, NOT ESTIMATED · READ FROM THE BUILT .SO

```
cuobjdump -res-usage build/aare/_aare_cuda*.so | c++filt
3x3: REG:38 STACK:0 LOCAL:0 # STACK/LOCAL 0 = no spills
9x9: REG:128 STACK:0 LOCAL:0
```

PER SM · SM_89 · F32 BUILD

3×3 · BLOCKS RESIDENT

6

9×9 · BLOCKS RESIDENT

2

SPILLS, EITHER CASE

0 bytes

3×3 ON THE F64 BUILD

47 regs → 5 blocks

How many blocks fit is the whole hardware story. The next slide is what that buys.

Occupancy is a latency-hiding budget

- When a warp stalls on memory the SM switches to another warp that is already resident. **Occupancy = resident warps / the maximum the SM can hold**: how many alternatives it has to switch to. More resident warps, more stalls hidden.

BLOCK SIZE

16 × 16

THREADS / BLOCK

256

THREAD SLOTS / SM

1 536

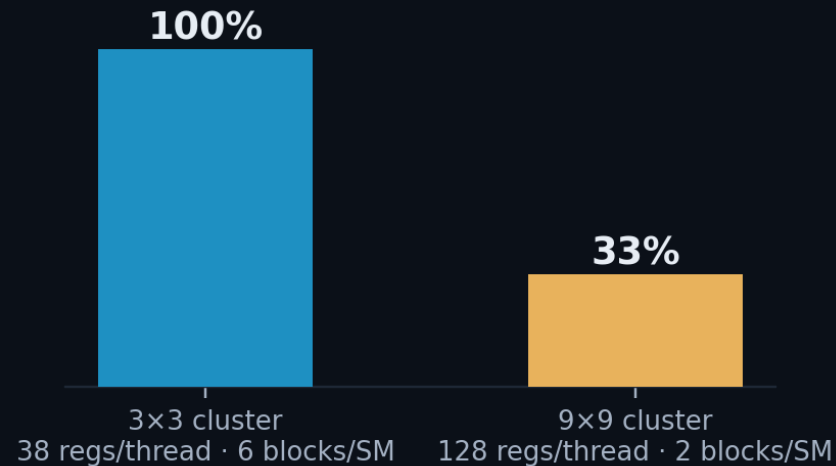
MAX WARPS / SM

48

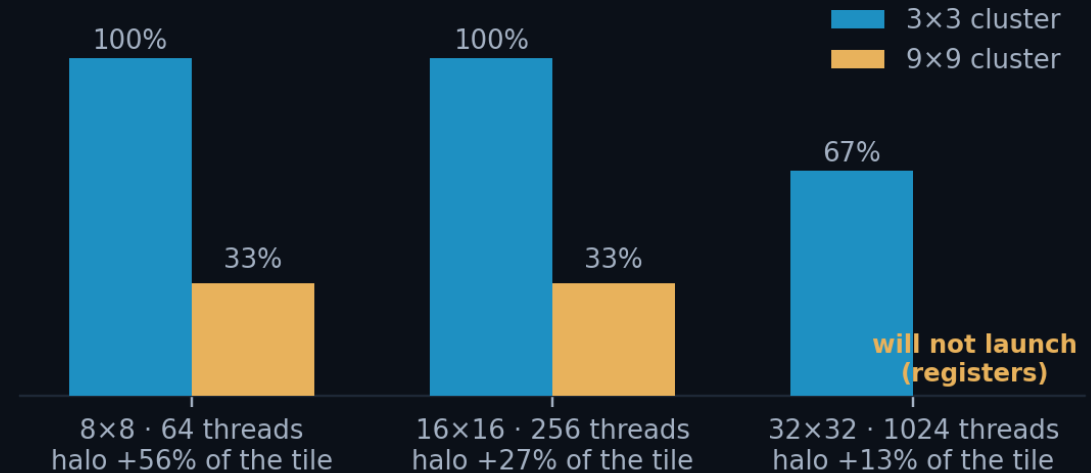
OCCUPANCY 3×3 · 9×9

100 % · 33 %

achieved occupancy, 16×16 block · f32 build



occupancy vs block size (halo overhead quoted for 3×3)



16×16 is the balance point: enough threads to amortise the halo, few enough that 6 blocks still fit. But 33 % is not a failure to fix: it is what the register budget allows, and at 9×9 one kernel nearly fills the machine on its own.

Three acts, ordered by which bar is tallest

ACT I · FEED THE GPU

the host cannot submit work fast enough at 3×3 [f64]

opt1	First CUDA port 1 stream, one launch per frame	×2.34
opt2	Streams + batching 4 streams, 2 000-frame batches	×3.66
opt3	Pipeline rework sync barriers removed	×4.32
opt4	Pinned memory DMA-speed host transfers	×5.69

ACT II · GET THE RESULTS BACK

the host copy is now the tallest bar at 3×3 [f64]

opt5	Host↔GPU overlap chunked submit / collect	×7.45
opt6	Zero-copy collection read in place, never copy	×8.65

ACT III · THE KERNEL

the kernel is the tallest bar, at 9×9 at 9×9 [f32]

opt7	FP32 pedestal + variance rewrite the only kernel change in the deck	kernel −41%
------	---	-------------

Speedups are 3×3 vs the best CPU configuration, 24 threads. Three routes were measured and rejected: CUDA Graphs, and two faster ways of copying the results. All three are in annex A2, because the rule that predicts the wins predicts the failures too.

ACT I OF III · FEED THE GPU

The host cannot submit work fast enough

The kernel was fast almost immediately. This act is about the host, and it is told at 3×3, where PCIe is the floor.

STARTING FROM

6 762 FPS

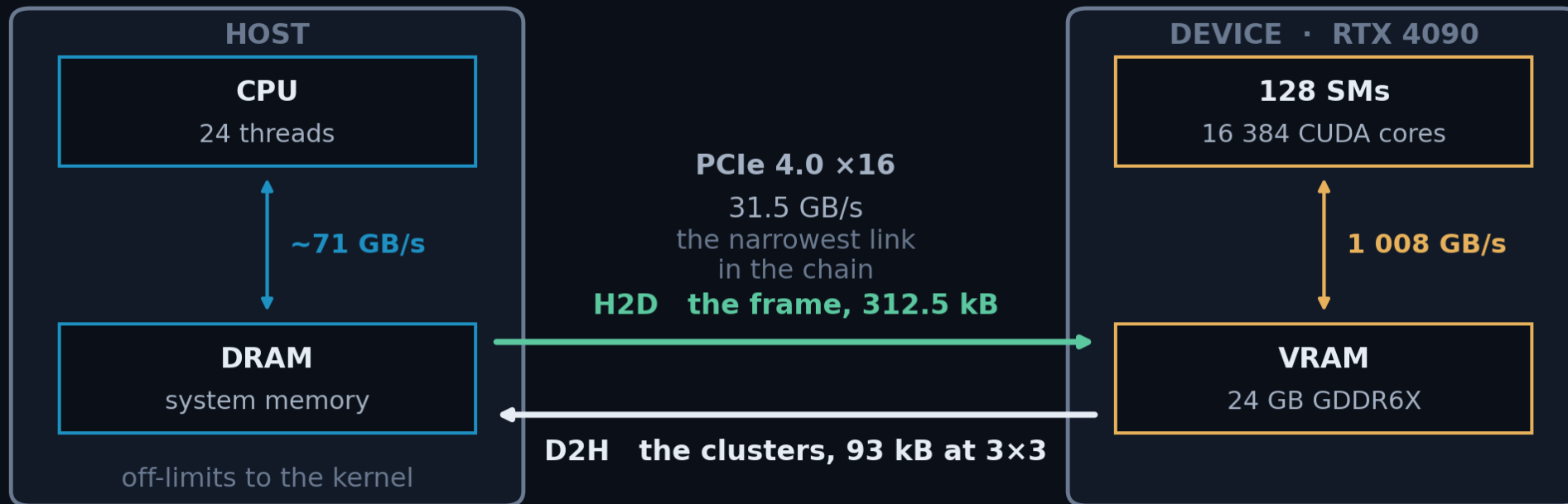
24-thread CPU · 14.8 s for 100 000 frames

COMING UP

- 12 two memories, one wire
- 13 opt1 · first port
- 14 opt2 · streams + batching
- 15 opt3 · no barriers
- 16 opt3 · one D2H, not two
- 17 opt4 · pinned memory

The GPU consumes 32× faster than PCIe delivers

- A CUDA kernel addresses **the GPU's own memory and nothing else**. So every frame is copied in — **H2D, host to device** — and every result copied back out — **D2H**. Those two names are what the next five slides are about.



The asymmetry is the whole talk. Host DRAM runs at ~71 GB/s and VRAM at 1 008, but everything between them crawls through **31.5**. Arithmetic is cheap once the data is there; arriving is not.

At 3x3, in numbers: the kernel needs **5.5 μs** per frame. Getting that frame across PCIe costs **13.2 μs** uncontended. The link wins before the kernel has done anything.

The first port runs at 26 % of the GPU's floor

- Shared-memory tiling with **halo loading** for any cluster size; pedestal subtraction fused into the tile load.
- Cluster geometry is a **compile-time template parameter** → the 3×3 stencil is fully unrolled.
- One cudaMemcpy in, one kernel, one cudaMemcpy out; **the host blocks on every frame.**

ClusterFinderCUAOpt2.hpp · find_clusters()

```
// one frame at a time, the host waits at every step
cudaMemcpy(d_frame, h_frame, bytes, cudaMemcpyHostToDevice);
find_clusters_in_single_frame<ClusterType, FRAME_TYPE>
    <<<grid, block, shmem>>>(d_frame, d_pd_mean, ...);
cudaMemcpy(h_out, d_out, out_bytes, cudaMemcpyDeviceToHost);
```

PCIe is full-duplex: H2D, kernel and D2H run on independent engines and overlap, so the **floor** — the fastest a frame can go if the host cost nothing — is **max(H2D, kernel, D2H)**, never the sum. At 3×3 that is $\max(16.17, 15.17, 7.69) = 16.2 \mu\text{s} \rightarrow 61\,859 \text{ FPS}$. Exactly how each of those three is measured is slide 21; it does not change this one.



OPT1 · 3×3 · 100 K FRAMES · F64

THROUGHPUT

15 807 FPS

VS 24-THREAD CPU

×2.34

PER FRAME

63.3 μs

THE GPU FLOOR

61 859 FPS

% OF FLOOR

26 %

Four streams and 2 000-frame batches: ×1.56

- A **stream** is an ordered queue of GPU work. Work in **different** streams may overlap, so a copy can run while another stream computes.
- Each stream gets its own **StreamContext**: device frame buffer, output buffer and pedestal. Frames are handed out **round-robin**.
- The host now submits **2 000 frames per call** instead of one.

ClusterFinderCUDA.hpp · per-stream state

```
struct StreamContext {  
    cudaStream_t    stream;  
    FRAME_TYPE      *d_frame;    ClusterType *d_clusters;  
    PEDESTAL_TYPE    *d_pd_mean, *d_pd_sum, *d_pd_sum2;  
};  
auto &sc = v_sc[frame_idx % n_streams]; // round-robin
```



OPT2 · 3×3 · 4 STREAMS · BATCH 2 000

THROUGHPUT

24 726 FPS

VS 24-THREAD CPU

×3.66

PER FRAME

40.4 μs

STEP GAIN OVER OPT1

×1.56

% OF FLOOR · 61 859 FPS

40 % (was 26)

Scaffolding, not yet the payoff. The streams exist, but the host still synchronises after every round: see opt3.

One sync per batch, not one per round: ×1.18

- opt2 synchronised **all streams after every round** of $n_streams$ frames. The GPU drained to empty each time.
- opt3 submits every frame's H2D → kernel → D2H **asynchronously**, then synchronises **once at the end of the batch**.

opt1 · 1 stream, synchronous

■ H2D copy ■ kernel ■ D2H copy



opt2 · 4 streams, sync barrier per round

barrier — H2D starves



opt3 · 4 streams, barriers removed



streams never wait on each other — the H2D engine never goes idle
time →

THE ONE-LINE IDEA

```
// opt2: barrier after every round
for (round) {
    submit(n_streams frames);
    cudaDeviceSynchronize();
}

// opt3: submit everything, sync once
for (frame : batch) {
    cudaMemcpyAsync(..., sc.stream);
    kernel<<<..., sc.stream>>>(...);
    cudaMemcpyAsync(..., sc.stream);
}
for (sc : streams)
    cudaStreamSynchronize(sc.stream);
```

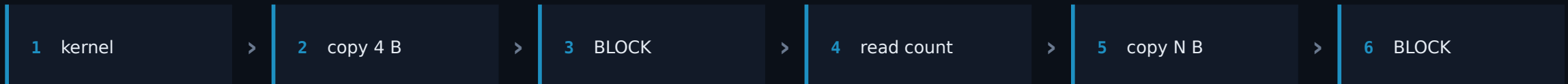
29 188 FPS · ×4.32

34.3 μs/frame · 47 % of floor (was 40)

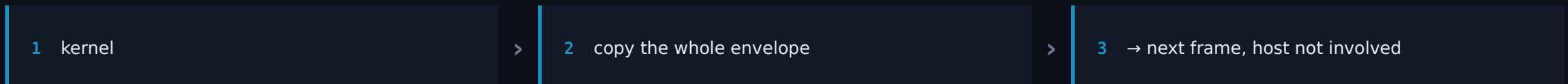
Each lane is one stream. Removing the barrier lets a stream start its next frame while its neighbours are still copying. The three panels are scheduled, not sketched: H2D and D2H are one FIFO engine each, so a stream waits for the copy engine, never for another stream's copy to finish overlapping it.

One D2H per frame, not two

- opt2 asked the device **how many clusters**, blocked until the answer came back, then asked for **that many**. The size of the second copy was a function of data that had not arrived yet.
- opt3 gives every frame a **fixed envelope** — count, then room for **cap** clusters — so the copy's size is known at construction and can be queued with the kernel. The count is still read, but **afterwards**, on the host.



opt2 · two transfers and two stalls per frame, because the second one cannot be issued until the first has landed.



opt3 · one transfer, no stall. Nothing in the loop waits on a value.

You cannot stream a transfer whose length depends on the transfer before it. opt3 pays bytes to delete that dependency: the envelope is sized by the **cap**, not by how many clusters were found, so an empty frame costs the same D2H as a full one: 120 kB at 3×3 against 93 kB of real clusters.

Everything downstream needs that fixed layout: opt6 could not hand out a view into a buffer whose shape was not known in advance.

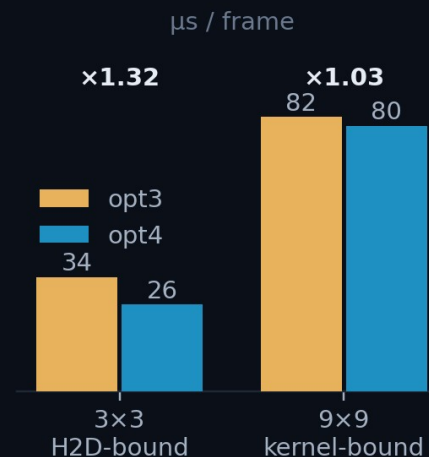
Pinning the input buys DMA-speed H2D: $\times 1.32$

- Normal host memory is **pageable**: the OS may move or swap it. A DMA engine cannot safely read that, so the driver first copies your data into a **hidden pinned staging buffer**. Every transfer is copied twice.
- Pinning** locks the pages in physical RAM. The GPU's DMA engine then reads host memory **directly**, no staging copy, and the transfer can be truly asynchronous.

PAGEABLE · before opt4



PINNED · opt4



ClusterFinderCUDA.hpp

```
// pin the whole dataset once
cudaHostRegister(ptr, bytes,
                  cudaHostRegisterDefault);

// ... run the whole campaign ...

cudaHostUnregister(ptr);
```

38 486 FPS · $\times 5.69$
26.0 μs/frame · 62 % of floor, the largest step in Act I

Measured H2D [s1, uncontended]: one 400×400 uint16 frame (312.5 KiB = 320 000 B) in 13.2 μs = 24.2 GB/s, 77 % of PCIe 4.0 ×16 theoretical, i.e. true DMA speed. In the shipped pipeline it reads 16.6 [s4], +26 % of H2D↔D2H contention (A1). Pageable staging runs ~15 GB/s.

The rule, first sighting: $\times 1.32$ at 3×3 but only $\times 1.03$ at 9×9. Pinning attacks H2D, the tallest bar at 3×3, the shortest at 9×9.

ACT II OF III · GET THE RESULTS BACK

The host copy is now the tallest bar

Both transfers are DMA already. What is slow is what happens AFTER the D2H: copying clusters out of the finder's pinned buffer into heap the caller owns. Host to host. Still 3×3, but 9×9 is where this act pays most.

ARRIVING AT

38 486 FPS

opt4 · 26.0 μ s per frame · 62 % of the GPU floor

COMING UP

- 18 opt5 · host↔GPU overlap
- 19 9×9 · why overlap runs out
- 20 opt6 · zero-copy

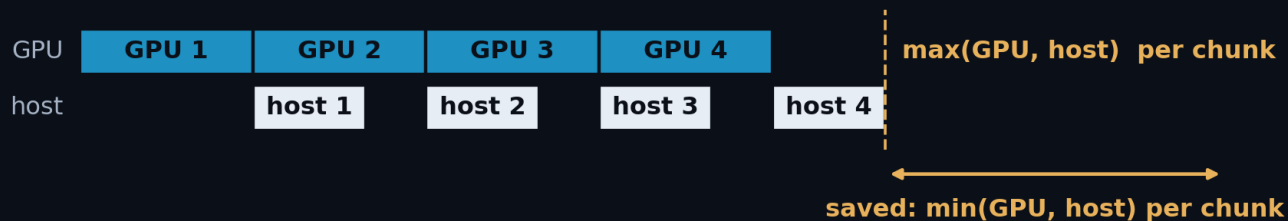
Overlapping host and GPU hides $\min(\text{host}, \text{GPU})$: $\times 1.31$

- opt3 overlapped H2D || kernel || D2H **across streams, inside one batch**, but never the **host** with the GPU: `find_clusters_batched` synchronised, then built thousands of ClusterVectors with the GPU idle.
- opt5 keeps **one batch in flight while materialising the previous one**: chunk $i+1$ is submitted before chunk i is collected.

submit → collect, serialized (opt4) · 3×3



submit($i+1$) before collect(i) (opt5)



you never write these: `find_clusters_batched()` wraps them

```
tok = cf.submit_batch(data[a0:b0], first_frame=a0)
for a, b in bounds[1:]:
    nxt = cf.submit_batch(data[a:b], first_frame=a) // GPU starts i+1
    results.extend(cf.collect(tok)) // host unpacks i
    tok = nxt
```

OPT5 · 3×3 · NO CUDA WORK AT ALL

3×3 THROUGHPUT

50 410 FPS

PER FRAME · STEP · OF FLOOR

19.8 μ s · $\times 1.31$ · 81 %

THE WHOLE CHANGE

CUDA API CALLS ADDED

none

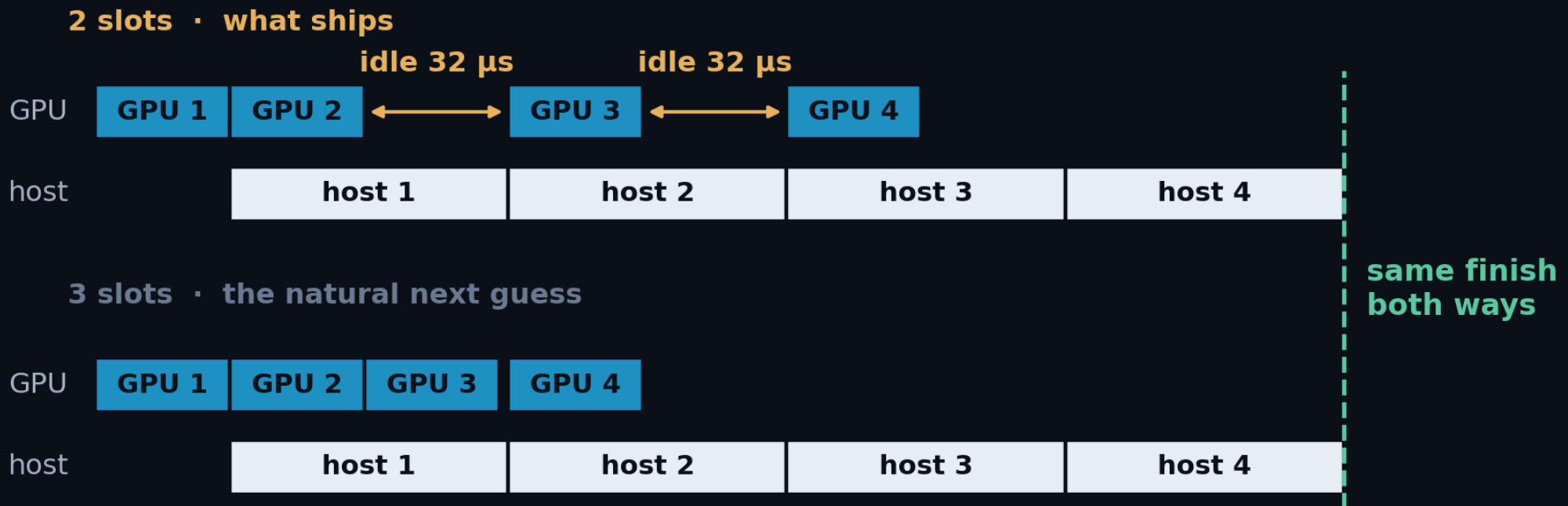
WHAT MOVED

the host loop

Told at 3×3 , where the host copy is SHORTER than the GPU floor and tucks underneath it. 9×9 is the other case, and it is the next slide. For clusters that must outlive the finder, 3×3 opt5 is the endpoint: opt6 lends, it does not give.

Overlap runs out: the host is the taller bar

- At 3×3 the host copy is **shorter than the GPU floor** and hides underneath it. At 9×9 it is **roughly twice the floor** — ~62 μ s of malloc-and-copy against 30.01 μ s of GPU — so overlap still works, it just has less to hide. That is why opt5 is worth **×1.20** here and ×1.31 at 3×3.

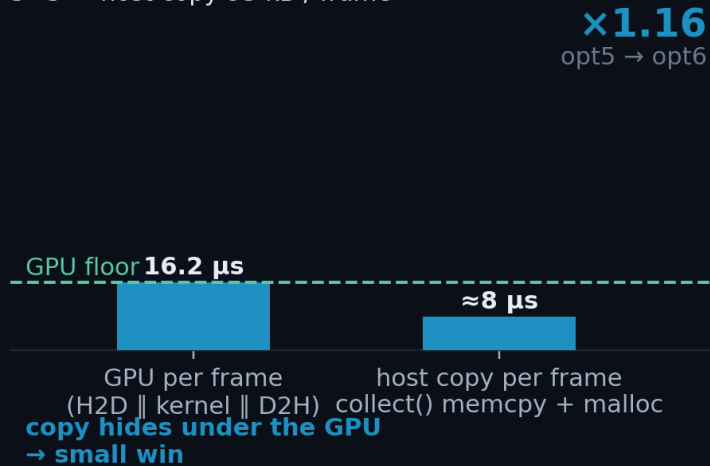


A deeper buffer **relocates the GPU's idle**, it **does not close it**: the host lane is already back-to-back in both strips, so it alone sets the pace. **The only way down is to make the host term smaller.**

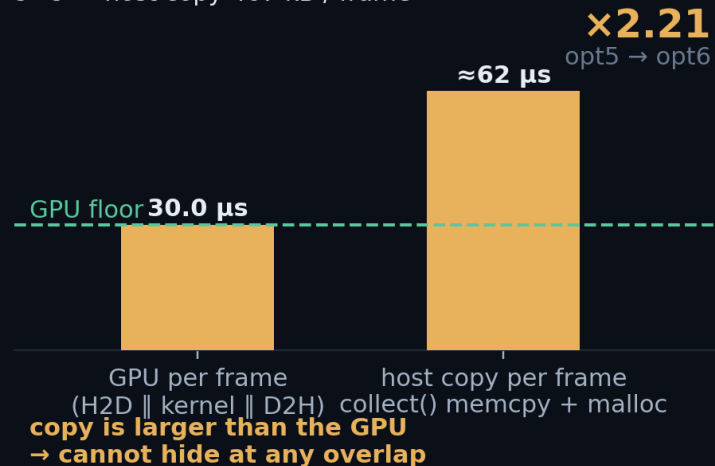
Read the results in place: ×2.21 at 9×9

- The D2H lands in a **pinned host buffer**. `collect()` then allocates one `ClusterVector` per frame and `memcpy`s into it; at 9×9 that is **467 kB per frame**, **~9.3 GB per run**, single-threaded.
- `collect_view()` returns a **BatchView**: strided numpy views straight onto the pinned buffer. It withholds ownership past the chunk, **not access**, every cluster's payload and coordinates are readable.

3×3 · host copy 93 kB / frame



9×9 · host copy 467 kB / frame



OPT6 · 3×3 AND 9×9 · COLLECT_VIEW()

3×3 THROUGHPUT

58 495 FPS

PER FRAME · STEP · OF FLOOR

17.1 μs · ×1.16 · 95 %

9×9 THROUGHPUT

33 323 FPS

PER FRAME · STEP · OF FLOOR

30.0 μs · ×2.21 · 100 %

opt5 → opt6: 19.8 → 17.1 and 66.4 → 30.0 μs, bit-identical, 0.2 % spread, zero warm faults. FLOOR = lower of the s4 engine max and the best sustained rate: at 3×3 the 16.17 μs max sets it; at 9×9 f64 the run BEATS the 32.66 μs max, so the 30.0 sustained sets it [A1].

The two bars are the competing costs, not the two steps; the step times are on the right.

ACT III OF III · THE KERNEL

Only now is the kernel the tallest bar

The story moves to 9×9, where the kernel is finally the tallest bar. First, how the engine times in this act are measured.

ARRIVING AT

58 495 FPS

opt6 · 3×3, and 33 323 FPS at 9×9, where this act pays

COMING UP

- 21 how the engine times are measured
- 22 opt7 · FP32 pedestal
- 23 catastrophic cancellation
- 24 why it comes last

Two configurations, one floor

s1

One stream, nothing else running

How long an operation actually takes. The right number for a capability claim, and for the headroom that is left.

s4

The shipped pipeline, four streams

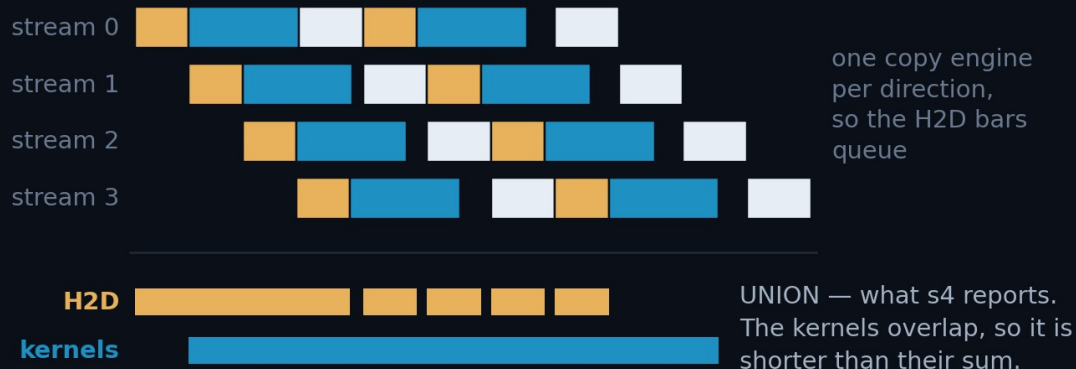
How BUSY each engine is per frame: the union of its intervals, which is not the sum of the durations. The only number that can set a floor.

floor

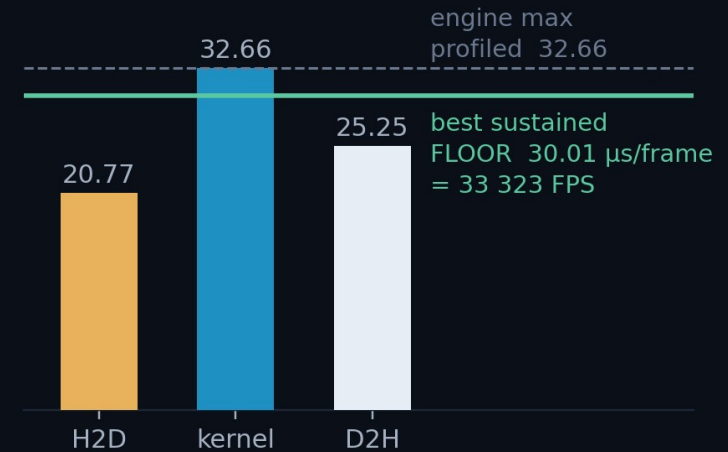
Set by the busiest engine

The fastest a frame could go if the host cost nothing. In μs per frame, or its reciprocal in FPS. The LOWER of the profiled estimate and the best rate sustained.

s4 · the shipped pipeline, four streams · schematic, 9×9 shape



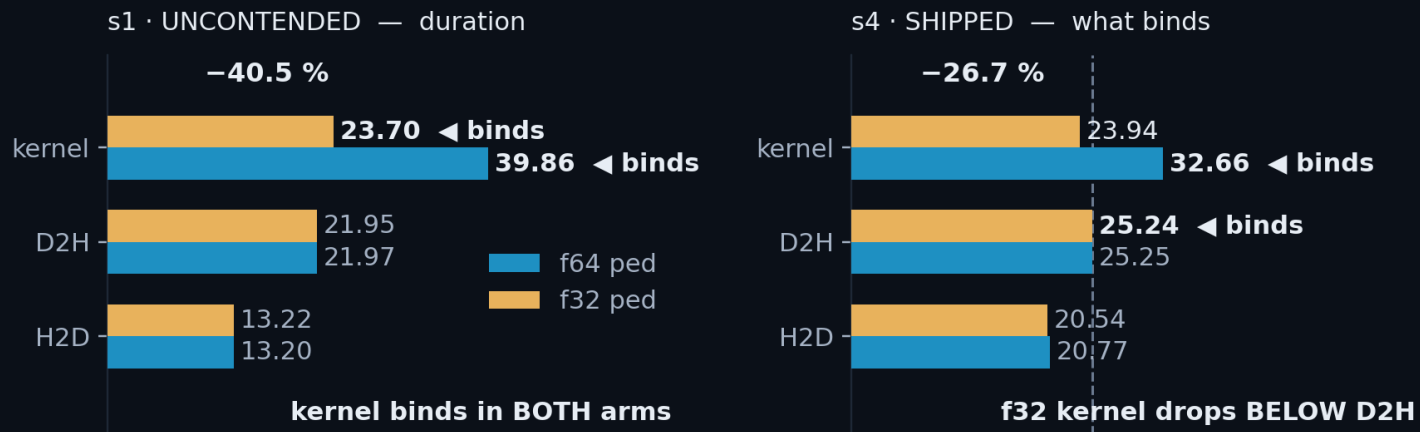
busy μs / frame · 9×9 f64 · measured



Nothing that measures a duration falls under load, yet the 9×9 kernel row falls 39.9 → 32.7 μs from s1 to s4. That is the tell: s4 is occupancy, not duration. The floor is quoted both ways in this deck: 30.01 μs per frame is 33 323 FPS. Full engine grid: **annex A1**.

FP32 halves pedestal traffic: –41 % kernel time

- ~80 % of pixels take the **pedestal-update** branch, which reads and writes six accumulator values. All four pedestal arrays are `DEVICE_PED_TYPE`, so one typedef halves every one of them: **48 bytes per updating pixel in FP64, 24 in FP32**. The kernel is **bandwidth-bound**, so halving the traffic nearly halves the time.



ONE TYPEDEF

```
// clusterfinder_kernel.cuh
using COMPUTE_TYPE    = float;
using DEVICE_PED_TYPE = float;
//                    was: double
```

Kernel, 9×9 [s1 · cap 1700]
39.86 → 23.70 μs (–40.5 %)

At 3×3 the same typedef is worth **–70.6 %** in the kernel and **4.6 %** end to end: there the kernel was never the tallest bar.

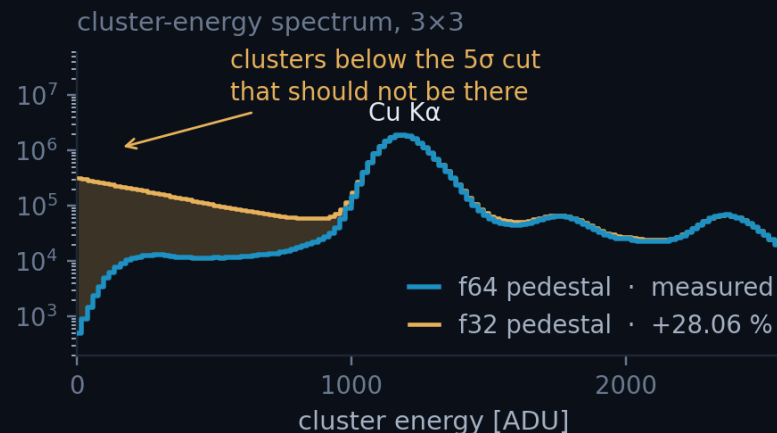
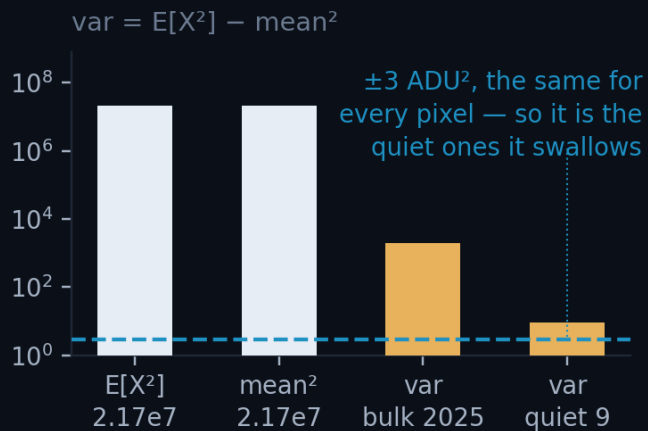
Naive FP32 is **wrong** (next slide).

Two more effects, both real, neither the reason it works: on a GeForce part FP64 arithmetic runs at 1/64 of FP32, and the pedestal update was paying that on every pixel; and the narrower accumulators free 9 registers at 3×3 (47 → 38), buying back a block per SM.

Both builds, same git rev, 20 000 frames. Full engine grid: A1.

Accumulate what is small, not what is large

- **The trap.** $\text{var} = E[X^2] - \text{mean}^2$ subtracts two numbers near 2×10^7 to get one near 2 000. FP32 carries ~ 7 digits, so the answer inherits an **absolute** error that **does not shrink as the answer does**.
- **What it cost.** On a quiet pixel that error is a third of the variance; a little lower it goes negative, the rms **clamps to zero**, and a 5σ gate becomes a **0σ gate**. Those pixels fire every frame, below the physical threshold.
- **The fix.** Freeze $X_0 = \text{round}(\text{mean})$ at the end of pedestal training and accumulate the **centred** value $Y = X - X_0$. Both operands become $O(\text{rms})$; the cancellation is gone.



Left: the two operands and the answer, log scale, against the $\pm 3 \text{ ADU}^2$ error. Right: the f64 curve is measured (23.2 M clusters); the f32 curve is reconstructed: measured area, modelled shape, method in the notes. The two-line patch itself is in annex A5.

NAIVE F32 · WHAT IT DID

EXTRA CLUSTERS

+28.06 %

PIXELS AFFECTED

~1-2 % of the sensor

OPERANDS VS ANSWER

2.17×10^7 vs $\sim 2\,000$

ERROR FLOOR

$\pm 3 \text{ ADU}^2$, absolute

AFTER THE REWRITE

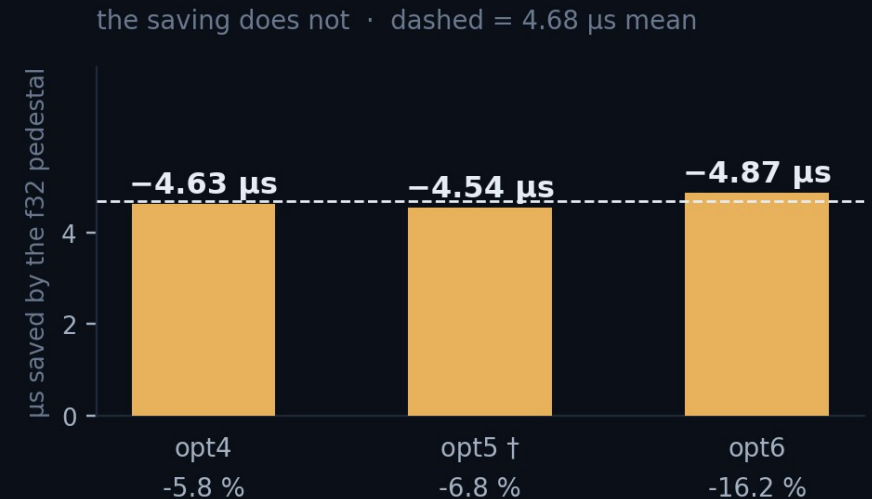
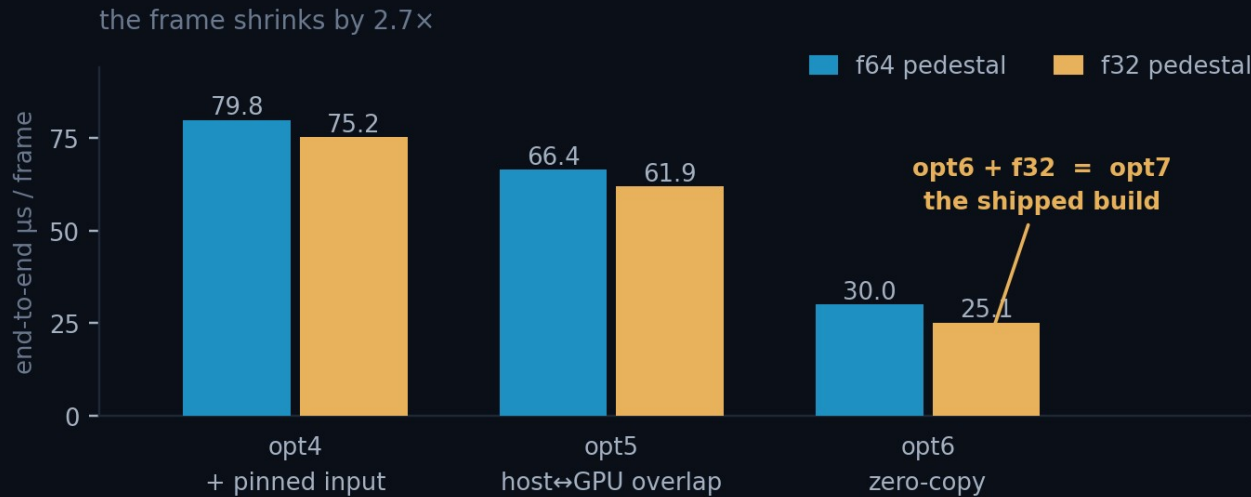
F32 VS F64 COUNTS

3×10^{-7}

VS THE CPU BASELINE

0.0039 %

The saving never grew — the frame around it shrank



The saving never grew: -4.63 μs at opt4, -4.87 μs at opt6. What shrank is the frame around it, 79.8 → 30.0 μs, so the same microseconds read **-5.8 %**, then **-16.2 %**. Act II made the kernel **separable**, not bigger.

And the act ends by handing the floor away. At **s4** the f64 arm is kernel-bound, **32.66** against a **25.25 μs** D2H. The typedef puts the kernel at **23.94**, below a D2H that never moved.

9×9 · 20 000 frames · 4 streams · cap 1 700 · warm · both arms at the same git rev. opt3 excluded and opt5 marked †: their arms sat in different allocator states. Worked through in annex A4.

RESULTS · WHAT CAME OUT OF IT

The whole ladder, and how to use it

Both cluster sizes end to end, and the audit behind the numbers.

ARRIVING AT

58 495 FPS

opt6 · everything after this is the ladder seen whole

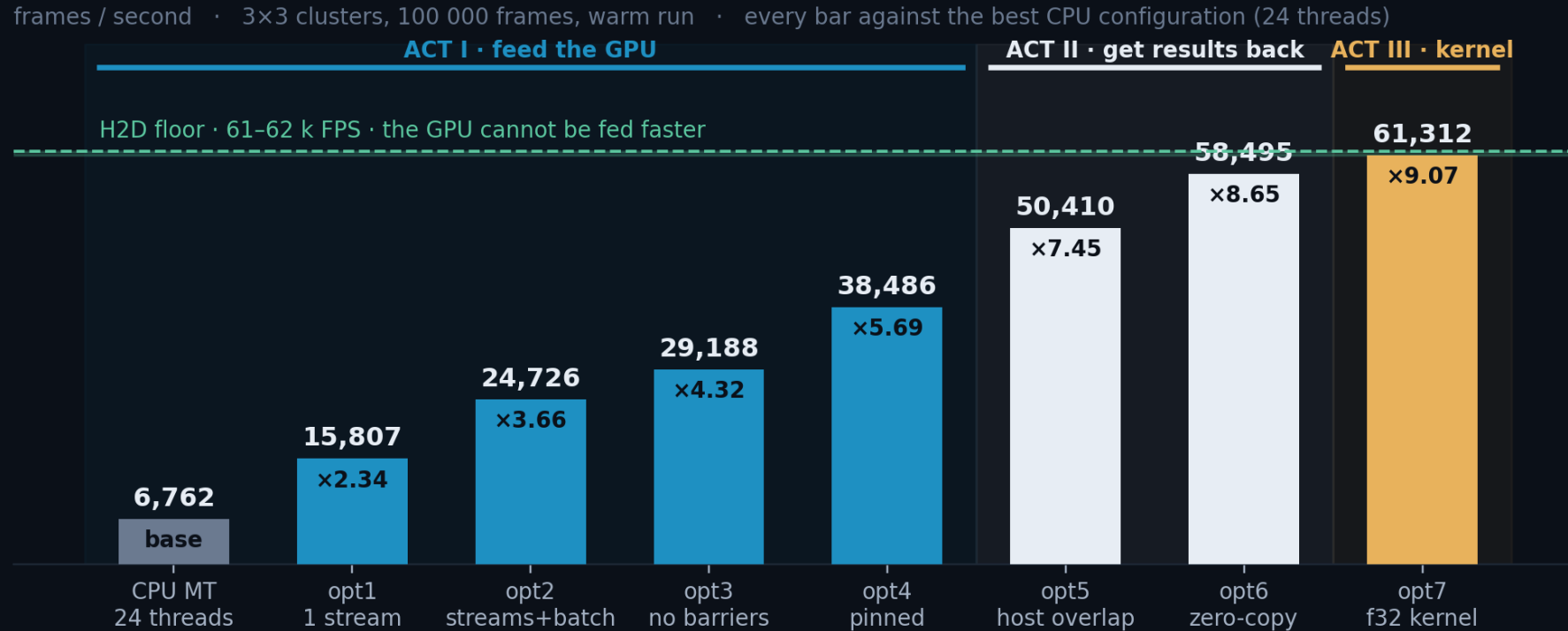
COMING UP

25–26 Results, both cluster sizes

27 Where the time went

28–29 What the numbers survived

×9.1 at 3×3, sitting on the H2D floor

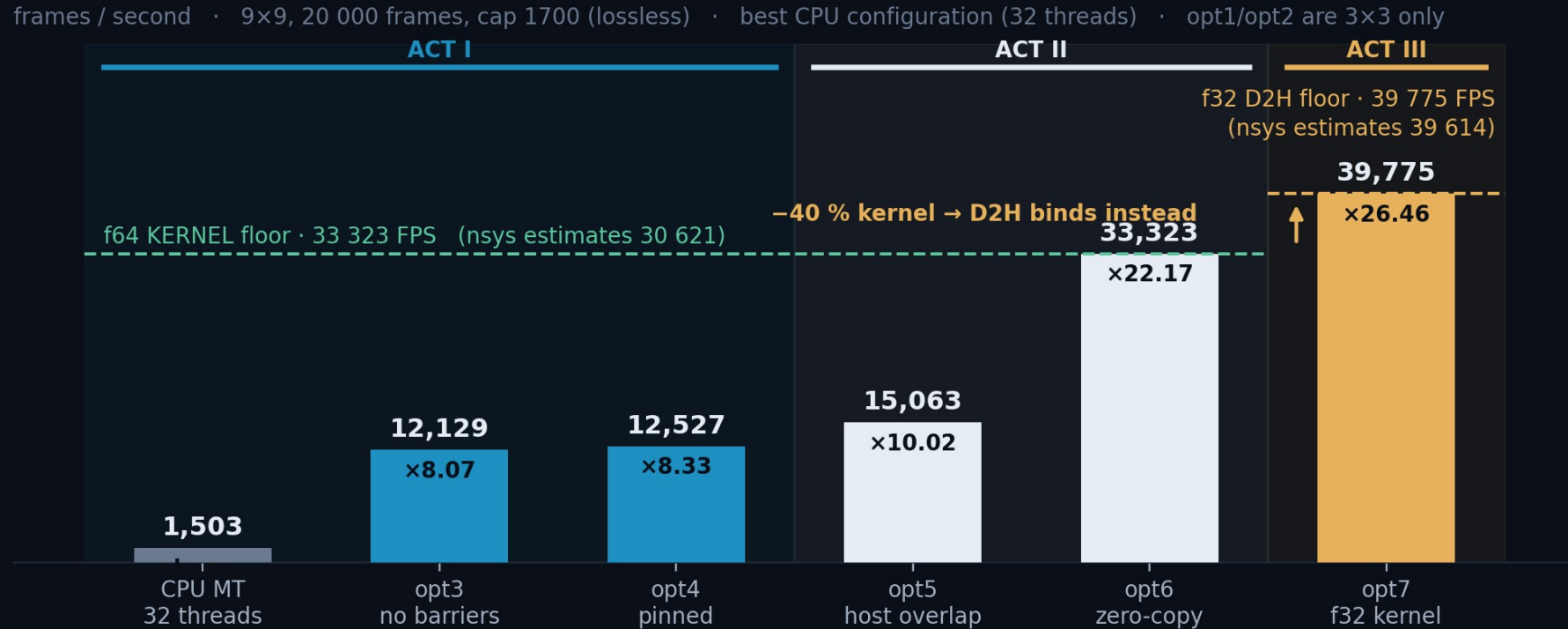


×9.1 over 24 CPU threads, 14.8 s → 1.63 s for 100 000 frames, and **at the H2D floor**.

Every step is **monotonic**, and correctness is held constant **throughout**: 0.004 % against the CPU baseline; against the CPU twin that isolates the port, **exact on the f64 pedestal** and **6 clusters in 23 M** on the shipped f32 (slides 30–33).

3×3 clusters · $\sigma = 5$ · 100 000 frames · batch 2 000 · 4 streams · 5 reps · warm = best of reps 1–4 (collect() does not converge, it oscillates between allocator states) · each step in its own process · CPU baseline = ClusterFinderMT at its best thread count, 24 here, first pass only.

×26.5 at 9×9, and opt7 hands the floor to D2H

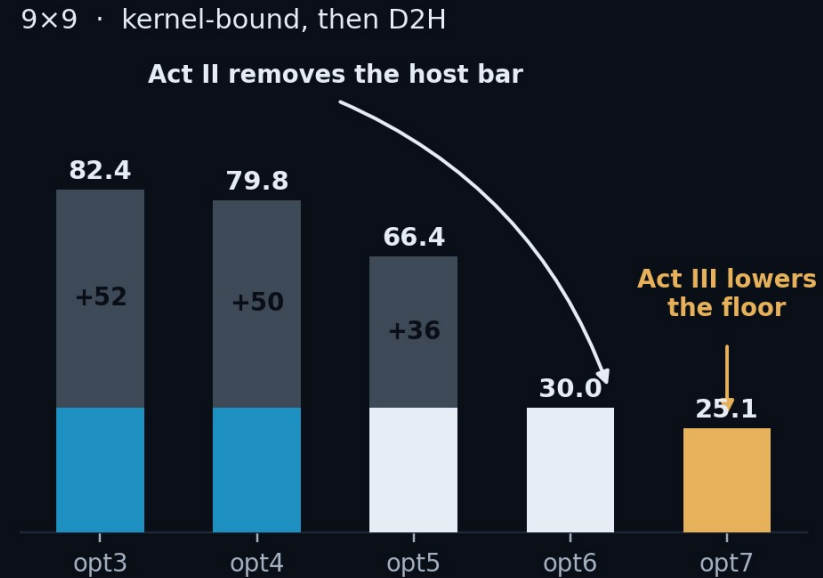
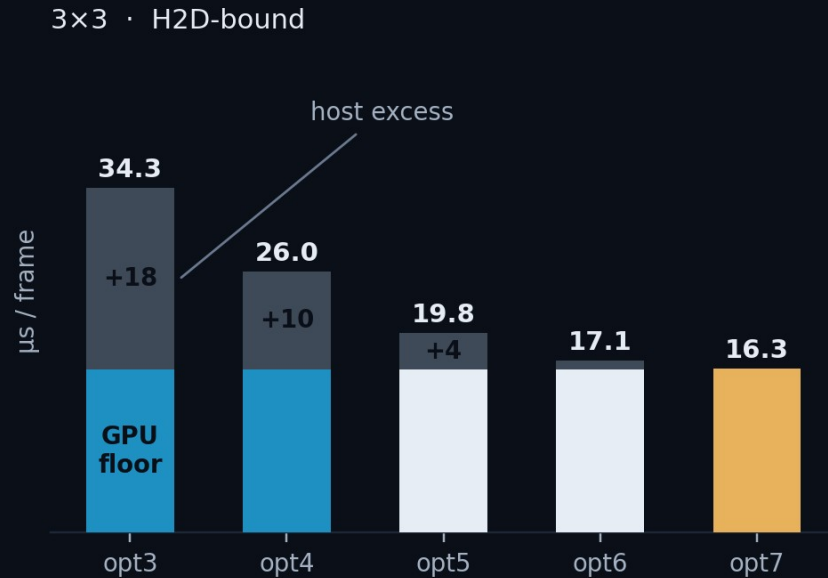


×26.5 over 32 CPU threads; the kernel is the tallest bar for the whole f64 arm, and opt7's -40 % drops it **below D2H**.

opt4 buys **×1.03** here and **×1.32** at 3×3. Same code, opposite regimes, the rule, twice.

9×9 · cap 1 700 (lossless; 1 500 truncated 0.0095 % of clusters) · 20 000 frames · opt1/opt2 are 3×3-only · CPU baseline ClusterFinderMT at 32 threads. The floor is the lower of the nsys estimate and the best sustained rate; both arms are sustained-bound, corroborated to 8.8 % (f64) and 0.4 % (f32). Why the arms differ: 544.5 kB of D2H costs 25.2 μs on both [s4], hidden under the 32.7 μs f64 kernel, not under the 23.9 μs f32 one.

The host bar dies first, then the floor itself drops



Acts I and II never touch the arithmetic. The GPU floor is a flat 16.2 μs at 3×3 / 30.0 μs at 9×9; what collapses is everything stacked on it.

Act III is the only step that lowers the floor itself, and it could not have been seen until the stack above it was gone.

Blue/white/amber = the GPU floor for that act's build: the LOWER of the s4 engine max (max of H2D, kernel, D2H; PCIe is full duplex, so never the sum) and the best sustained rate. At 3×3 the engine max sets it, 16.17 μs; at 9×9 f64 the sustained rate does, 30.01 against a 32.66 μs max. Grey = everything the host adds on top. At 9×9 the host contributes +50 μs at opt3 and nothing at opt6.

Materialising clusters costs 2.6 M page faults

- Every run that keeps its results **allocates a fresh heap and touches it once**. Linux only finds physical memory on **first touch**, so that first pass is where the OS does its work: **inside the timer**, on a run whose GPU work is under 2 s.

VIRTUAL · the ClusterVector malloc just handed you



already mapped

first touch → fault



PHYSICAL · 4 kB frames the kernel actually owns

MINOR FAULT · what the kernel does

- 1 traps into the kernel
 - 2 finds a free frame
 - 3 **zeroes all 4 kB of it**
 - 4 maps it; the write proceeds
- no disk: that would be a MAJOR fault

THE PROTOCOL · BRACKET EVERY TIMED CELL

```
# bracket every timed cell:
mf0 = resource.getrusage(resource.RUSAGE_SELF).ru_minflt
... t = time.perf_counter() - t0 ...
print(f'minor faults: {mf1-mf0:,}') # quote the run where this plateaus
```

ONE 100 000-FRAME PASS · 3×3

PAGES FAULTED IN

~2.6 M

COST INSIDE THE TIMER

up to 4 s

CLUSTERS MATERIALISED

~10 GB

COST PER FAULT, FITTED

0.68 μs

MAJOR FAULTS, ALL CAMPAIGN

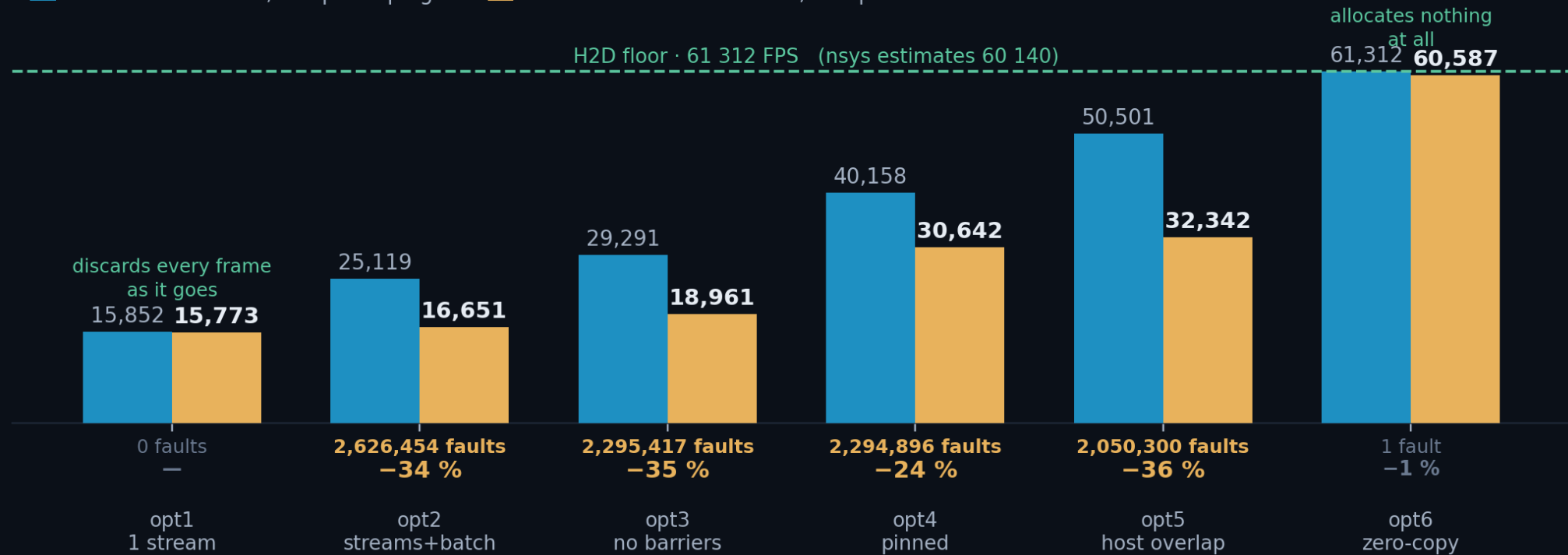
0

Not CUDA-specific: every anonymous allocation on Linux behaves this way. Copying faster cannot avoid it; not allocating can. That is opt6.

Re-run until getrusage() minor faults plateau (< 200 k), and quote that run. Validated: wall = steady-state + faults × 0.68 μs reproduced a 6.110 s run to within 1 ms.

A first run loses a third of its throughput to page faults

frames / second · 3×3, 100 000 frames, f32 · minor faults and the throughput they cost, per step
 ■ achievable · warm, 5-rep campaign ■ first run · results retained, one process



Everything that materialises clusters loses a third of its throughput on the first run, +7 to +20 μ s per frame, depending on how much of it the GPU can hide behind its own work.

Only the two ends escape, for opposite reasons. opt1 never grows the heap: it discards each frame. opt6 never needs one, and reaches **98 % of its peak on a cold process**.

Single pass, one process, every ClusterVector retained, f32, the same 100 000 frames. Not the campaign's "cold" rep, which discards results and so never grows the heap. Repeat the run and the amber bars climb onto the blue ones; opt1 and opt6 never move, because neither ever paid.

VALIDATION · DOES IT FIND THE SAME PHOTONS

Every number so far assumed the answers are identical

Whether the CUDA finder returns the same clusters as the CPU.

ESTABLISHED

×9.1 and ×26.5

on the hardware floor at both cluster sizes, if the physics holds

COMING UP

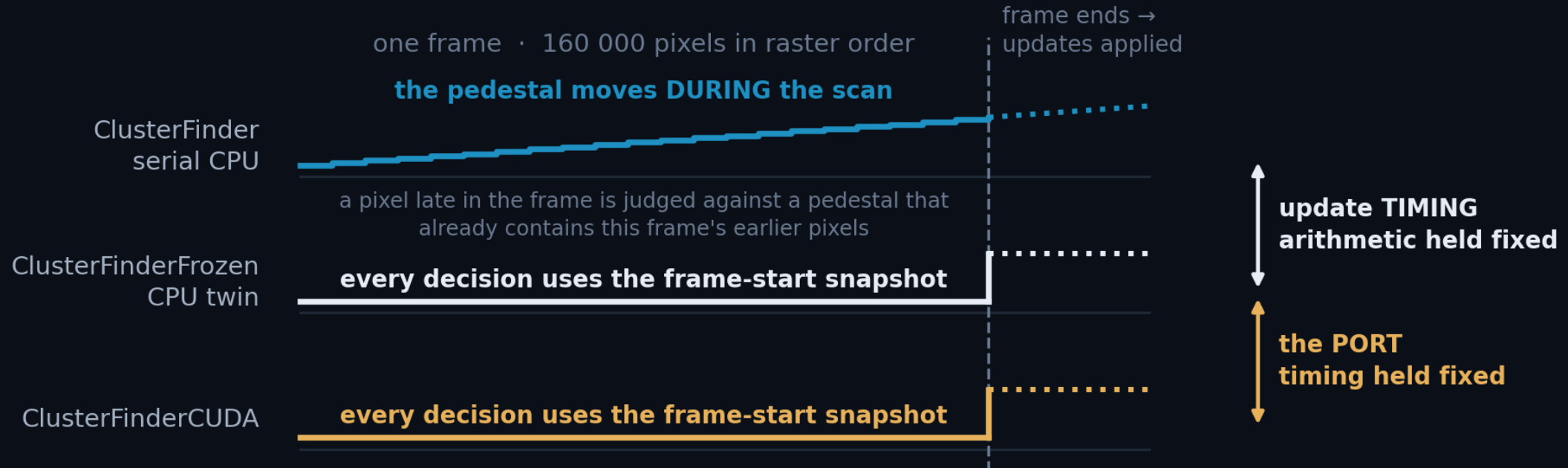
30–31 The fair comparison

32–33 The residual, dissected

34–35 For users

36 What is next

CPU and CUDA update the pedestal at different moments



The serial CPU finder updates the pedestal **as the raster scan reaches each pixel**. A CUDA thread cannot: 160 000 of them read the pedestal at once, so the update is **applied at the frame boundary**.

So a straight CPU↔CUDA comparison moves **two** things at once. **ClusterFinderFrozen** is the serial finder with only the update moved to the frame end: same arithmetic, same gates, same scan.

Frozen is a diagnostic twin, not a product: it exists so the next two slides can attribute each disagreement to exactly one cause. cpu vs frozen = update timing; frozen vs cuda = the port. Every finder on these slides is trained on the same 1 000 pedestal frames and run over the same 10 000 data frames.

CUDA and its CPU twin agree exactly: 0 in 23 million

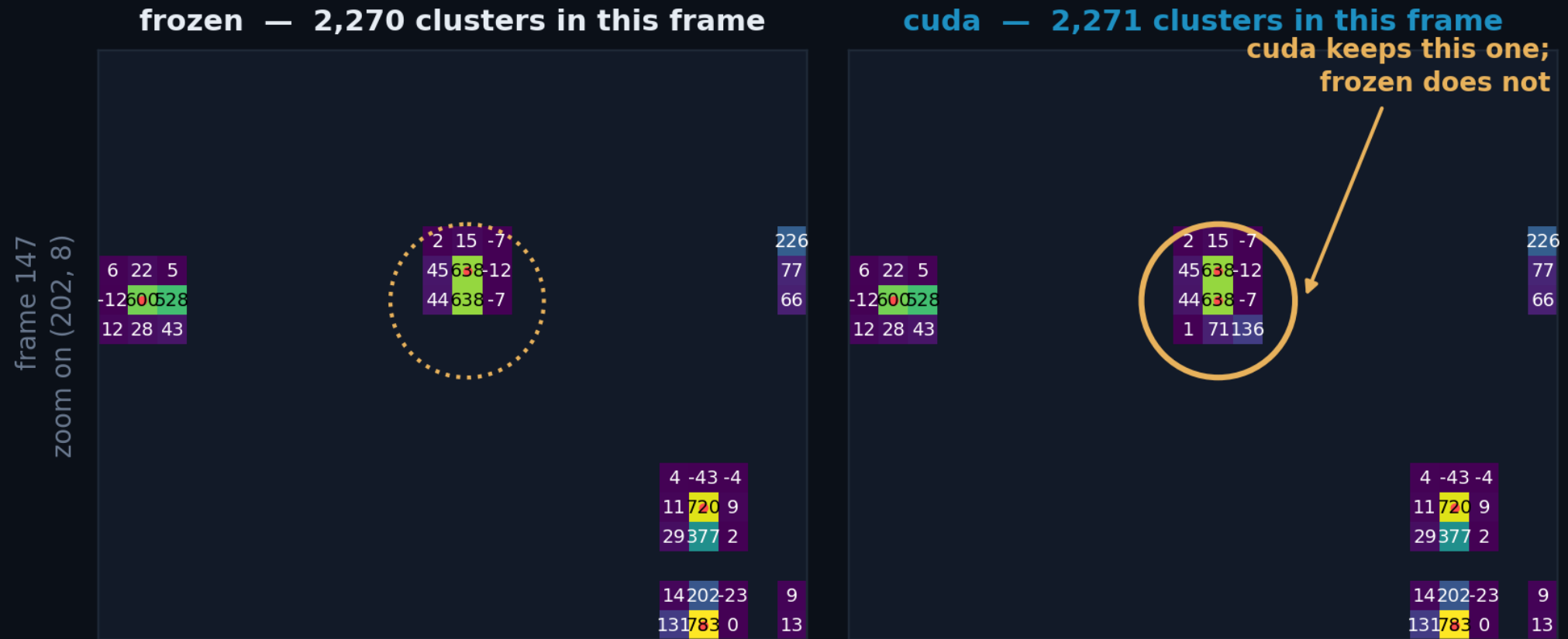
- **ClusterFinderFrozen** makes byte-for-byte the same decisions as ClusterFinder and differs in exactly one thing: **when** the pedestal is updated. Frozen per frame, pushed at frame end. That is the CUDA model, so comparing against it isolates everything else the port changes.

COMPARISON	THE ONE THING THAT DIFFERS	A-ONLY / B-ONLY	% OF CLUSTERS
serial CPU vs frozen CPU	update timing alone: a CPU-only effect	8 / 11	0.000082 %
serial CPU vs CUDA [f64 ped]	the same thing, and nothing else	8 / 11	0.000082 %
frozen CPU vs CUDA [f64 ped]	nothing	0 / 0	0 %
frozen CPU vs CUDA [f32 ped]	the float32 pedestal EMA drifting: see next slide	0 / 6	0.000026 %

The ringed pair is the port on its own, and at f64 it is identical: 23 244 605 clusters, not one disagreement. **cpu vs cuda** equals **cpu vs frozen** exactly, so everything CUDA changes is worth zero and the whole residual is a CPU-vs-CPU effect. The six at f32 are the next slide.

3×3 · 10 000 frames · 23.2 M clusters · same pedestal, same frames · exact centre-set difference at tol = 0. [f64 ped] and [f32 ped] are the same source built with DEVICE_PED_TYPE double / float; COMPUTE_TYPE is float in both, so the stencil arithmetic is identical across the two rows. Either row is reproduced by rebuilding with that typedef and re-running the comparison, which counts and localises every disagreement rather than only totalling them.

The whole disagreement is one duplicate centre



Only the **3×3 footprints of each finder's own centres** are drawn, so the panels differ **exactly** where the finders do. Cell values are pedestal-subtracted ADU.

cuda's patch is **one row taller**: a second centre directly below the one both found. **The charge is already counted**: a duplicate, not a new photon.

float32 cannot tell these two pixels apart

THE ONLY TEST THAT FLIPS, AND WHY

```
frame 147    centre (x=202, y=8)    3x3 window
raw window (ADU)    pedestal-subtracted, 1 decimal
[[4646 5282 4703]]  [[ 45.3 638.4 -12.1] frozen and cuda
[4857 5318 4950]    [ 43.7 638.4 -7.5] print the SAME
[4763 4640 4858]]  [ 1.1 70.7 136.4]] window
```

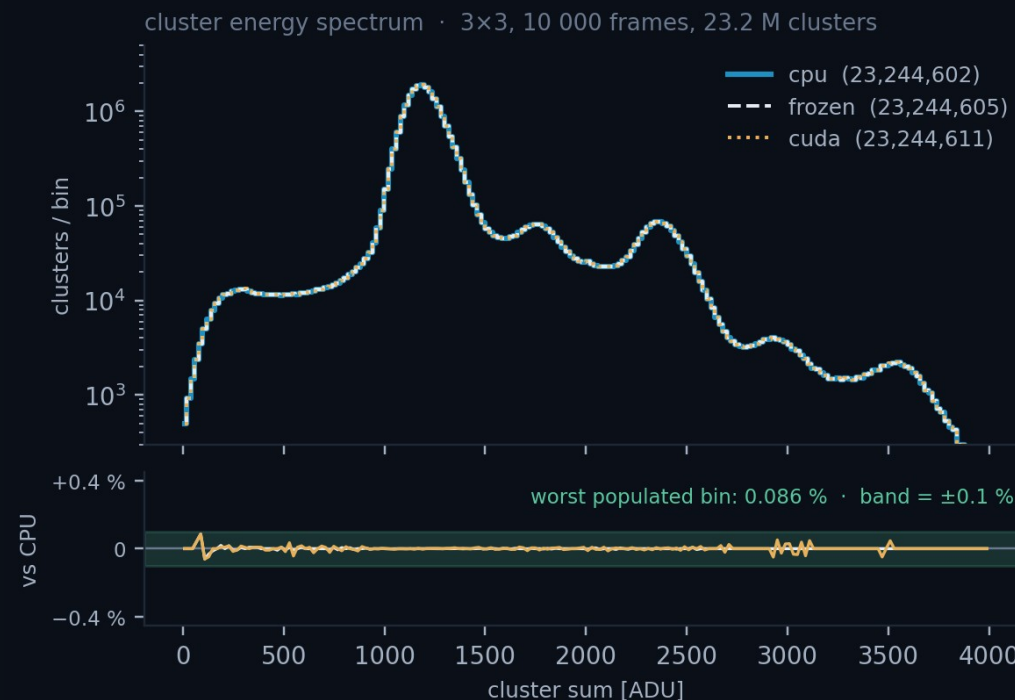
the two contenders, at full precision:

	rival (dy=-1)	centre
frozen [f64 ped]	638.383019956	638.382773664
cuda [f32 ped]	638.382812500	638.382812500

```
gate: accept if centre >= max(window)
frozen 638.382773664 >= 638.383019956 -> reject
cuda   638.382812500 >= 638.382812500 -> ACCEPT
```

Separation under the f64 pedestal: **0.000246 ADU**. One float32 ULP at 4 679.6 ADU is **0.000488**. The two pixels are **half a ULP apart**; in float32 they are *the same number*, and the gate accepts on a tie.

At f64 the residual is zero.



The same frame 147 as the previous slide, recomputed under each finder's decision-time pedestal. Both finders run the same gate (CPU value == max, CUDA !(val < max)), so the tie separates them, not the expression.

The fast path in eight lines

THE RECOMMENDED PATTERN

```
from aare import File, ClusterFinderCUDA

cf = ClusterFinderCUDA(image_size=(400, 400), cluster_size=(3, 3),
                        n_sigma=5, n_streams=4,
                        max_clusters_per_frame=3000)

for _ in range(1000):
    cf.push_pedestal_frame(pd.read_frame())

data = f.read_n(100_000)
cf.register_input_buffer(data)

for s in range(0, len(data), 2000):
    clusters = cf.find_clusters_batched(data[s:s+2000], first_frame=s)

cf.unregister_input_buffer()
```

ZERO-COPY · SAME PATTERN, STEP 4 SWAPPED

```
for v in cf.find_cluster_views_batched_iter(data, 2000):
    hist.fill(v.sums())
```

4, zero-copy
consume inside the loop:
the view dies with its chunk

- **3 · register_input_buffer()** is opt4 in one call. Pin **once**, outside the loop; slices of a registered array inherit it. Applies to both patterns.
- **4 · find_clusters_batched()** returns **one ClusterVector per frame**, in order, and runs opt5's chunked overlap internally, so you get it for free.

Zero-copy: ×1.16 at 3×3, ×2.21 at 9×9. The views expose every cluster; they only withhold **ownership** past the chunk.

Reduce as you go and you never allocate: histogram, sum, or copy the few you keep. Hold a view past its chunk and you stall the pipeline.

Steps 1, 2, 3 and 5 are unchanged. Only the call inside the loop differs, and with it who owns the memory the clusters live in.

Five knobs, and the one that silently truncates

PARAMETER	WHAT IT DOES	GUIDANCE
<code>n_streams</code>	How many frames may be in flight at once: an upper bound, not a count: the copy engines cap the real number below it.	4 at both sizes. 8 buys no kernel concurrency at 9×9 (+1% instance time) and inflates the CUDA-event timer 3.5×.
<code>max_clusters_per_frame</code>	Fixed size of the per-frame D2H transfer.	Must exceed the real maximum or clusters are silently dropped, and it sets the D2H bar directly. Measured at 9×9: the maximum is 1 633, and the lossless cap of 1 700 already makes D2H the bottleneck on the f32 build.
<code>batch_size</code>	Frames per <code>find_clusters_batched</code> call.	2 000 amortises launch overhead without a large pinned footprint.
<code>cluster_size</code>	Compile-time stencil geometry.	3×3 and 9×9 are registered; 9×9 moves the bottleneck off H2D and onto the kernel on f64, and onto D2H once opt7 shortens it.
<code>register_input_buffer</code>	Page-locks the host array for DMA.	Always, if the data is already in RAM. Check the pinning budget first.

The single most common mistake: leaving **`max_clusters_per_frame`** too low. It does not error; it truncates, and every frame quietly returns the same count.

The bottleneck has walked from the host, to the GPU, to PCIe

DONE

×9.1 at 3×3, ×26.5 at 9×9

16.3 and 25.1 μ s/frame end to end, both sitting on their hardware floor. At 3×3 that is 58 495 FPS against MÖNCH03's 1.3 kHz standard frame rate: 45× the detector, and ~10× its optimised 3–6 kHz ceiling.

DONE

FP32 pedestal, safely

–40.5 % kernel, and correct, because the variance is accumulated on a frozen per-pixel offset instead of a raw second moment. Against the CPU twin that isolates the port the decisions are identical; the shipped f32 pedestal adds 6 duplicates in 23 million.

NEXT

3×3: transfer granularity

The 16.31 μ s sustained sits 3.16 μ s above the uncontended 13.15 μ s H2D rate [f32: s4 vs s1]: 2 000 separate 320 kB descriptors, plus 26% of H2D→D2H contention.

NEXT

9×9: the result path, not the kernel

At a lossless cap D2H already binds [f32 · s4]: 25.24 μ s against a 23.94 μ s kernel. More kernel work buys nothing until the D2H slot stops being cap-sized.

Every number in this deck is measured on one machine, one dataset, one git revision — and the annexes carry the reconciliation for each of them.

Annexes

EVERYTHING SO FAR

35 slides

the arc is finished; what follows answers questions

COMING UP

- A1** Every engine number, reconciled
- A2** The three rejected routes
- A3** opt5 · the overlap code
- A4** The fault model, tested
- A5** The variance rewrite in full
- A6** The three benchmark artefacts
- A7** Which test causes the CPU/CPU gap

Uncontended, or as the pipeline runs it

• Every engine time in this deck is tagged **[build · s1|s4]**. **s1** is one stream with nothing else running: what an engine does **on its own**, which is the right number for a capability claim and for the headroom that remains. **s4** is the shipped four-stream pipeline: each engine's **busy time per frame**, the union of its intervals, the only number that can set a floor.

3×3 · μS/FRAME	S1 F64	S1 F32	S4 F64	S4 F32	9×9 · CAP 1700	S1 F64	S1 F32	S4 F64	S4 F32
H2D	13.14	13.15	16.17	16.63	H2D	13.20	13.22	20.77	20.54
kernel	14.72	4.32	15.17	5.53	kernel	39.86	23.70	32.66	23.94
D2H	5.31	5.27	7.69	7.57	D2H	21.97	21.95	25.25	25.24
engine max [s4]	—	—	16.17	16.63	engine max [s4]	—	—	32.66	25.24
FLOOR = lower of max, sustained	—	—	16.17	16.31	FLOOR = lower of max, sustained	—	—	30.01	25.14

Two traps this table closes. (1) s4 is engine *occupancy*, not duration: the 9×9 kernel row **falls** 39.86 → 32.66 while every transfer rises, and nothing that measures a duration falls under load. (2) **the FLOOR is not the engine max**: the max is profiled and runs 2-8 % high, so the floor is whichever is lower, it or the best unprofiled sustained rate, which is why opt6 reports **30.01 μs against a 32.66 max**.

THE D2H SHIFT IS AN s4 PHENOMENON: at s1 the kernel binds in both arms (39.86 and 23.70 against a 21.97 / 21.95 D2H); only under four-stream contention does D2H climb to 25.24 and overtake the 23.94 f32 kernel, so any claim about which engine binds must be read from the s4 columns. Bandwidth arithmetic in the notes.

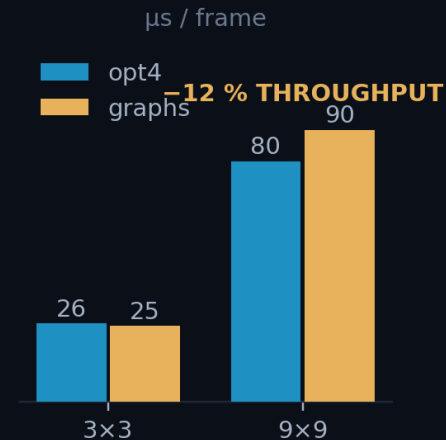
CUDA Graphs, a sound idea that the next act overtook

- Every cudaMemcpyAsync / kernel launch costs the **CPU** a few microseconds of driver work, per frame and per operation. After opt4 that looked like the budget.
- A **CUDA Graph** captures the whole dependency DAG once. Replaying it is a **single** cudaGraphLaunch: the driver already knows every node and edge.

WITHOUT GRAPHS · one driver call per operation, every frame



WITH GRAPHS · record once, replay with one launch



ClusterFinderCUDA_graph.hpp

```
// record once, at setup
cudaStreamBeginCapture(sc.stream, ...);
submit_h2d_kernel_d2h(sc);
cudaStreamEndCapture(sc.stream, &sc.graph);
cudaGraphInstantiate(&sc.graphExec, ...);

// per batch: one call
cudaGraphLaunch(sc.graphExec, sc.stream);
```

REJECTED

3x3: 39 752 FPS, inside noise of opt4.

9x9: 11 072 FPS, 12 % slower.

Its 3x3 edge was never established: the graph finder recorded no CUDA events while the stream finder did, and that instrumentation tax (2.8 μs) is larger than the gap (0.8 μs). More decisively, it never received the chunked pipeline of opt5, so it is competing on $\sim 2 \mu\text{s}$ of launch cost against a 24 μs floor. Launch overhead stops binding one step later; the technique aimed at it can no longer pay.

What a CUDA Graph actually saves, in microseconds

- The stream path issues **four runtime calls per frame**, a memset to clear the cluster counter, H2D, the launch, D2H. A graph replaces all four with **one** cudaGraphLaunch, so the ceiling on what it can save is $\frac{3}{4}$ of the submission cost.

CALL	PER FRAME	HOST COST	μS/FRAME
cudaMemcpyAsync	2	1.98 μs	3.97
cudaLaunchKernel	1	2.13 μs	2.13
cudaMemsetAsync	1	1.57 μs	1.57
submission total	4		7.67

THE ARITHMETIC

7.67 us/frame x 3/4 = 5.75 us/frame eliminated, AS MEASURED (under nsys)

5.75 / 4 (see A5) ~ 1.4 us/frame eliminated, unprofiled estimate

~1.4 μs against a 16.17 μs floor = 8.7 %, real while the host is the critical path, and **worth nothing after opt5**, which hides host work under the GPU entirely.

ROUTE A · MEASURED VERDICT

3×3 VS OPT4

×1.03

9×9 VS OPT4

×0.88

VS OPT5 AT 9×9

36 % behind

Read from CUPTI_..._RUNTIME, the host table, so this is the one order-of-magnitude number in the deck, ±50 %. It is enough to show graphs cannot pay against a 16 μs floor, and not enough to quote to two figures.

The copy is allocation-bound, not bandwidth-bound

B'

One allocation per chunk

`collect_packed()`

Removes the per-frame malloc but keeps the copy, and the copy is ~80% of the cost. Worse, the replacement allocation is 1.17 GB, far above glibc's mmap threshold, so it is mmap'd and munmap'd every chunk: 606 566 faults, ~21 µs/frame.

69.3 µs · deleted from the API

B''

Parallel materialisation

`8-thread copy pool`

Each worker gets its own glibc arena, which destroys the cross-run heap reuse that makes the single-threaded path cheap. Faults went 9 700 → 2 270 000; MALLOC_ARENA_MAX=1 collapsed them back to 138 k, which is the proof.

+6% at best, -33% when results are freed promptly

Copying faster does not help when the cost is the OS populating pages. The only winning move is not to allocate, which is exactly what opt6 does. `materialize_slot()` is deliberately single-threaded and carries a comment saying so, to stop the experiment being repeated.

The overlap, in six lines, and why you never write them

THE WHOLE OF OPT5

```
tok = cf.submit_batch(data[a0:b0], first_frame=a0)
for a, b in bounds[1:]:
    nxt = cf.submit_batch(data[a:b], first_frame=a) # GPU starts N+1 ...
    results.extend(cf.collect(tok))                 # ... host unpacks N
    tok = nxt
results.extend(cf.collect(tok))                     # drain the last one
```

- **You do not write this.** It is inside `find_clusters_batched()`, which chunks the batch and runs the loop for you, `opt5` arrived as a **speedup, not an API change**, and every existing caller got it without editing a line.
- `submit_batch()` and `collect()` stay public for anyone who wants the token by hand, streaming from a detector, interleaving other work between chunks.

Two chunks in flight is enough. A third adds pinned memory and no overlap: the host is already busy for the whole time the GPU is.

CHUNK SIZING · THE TWO CONSTRAINTS

MULTIPLE OF

`n_streams`

CAPPED AT

`MAX_SLOT_BYTES`

The chunk MUST be a multiple of `n_streams`. The device pedestal is per-stream and advances once per frame the stream sees, so an uneven chunk leaves the four pedestals at different ages, the finder would stop being reproducible, and two runs of the same data would not agree.

It is also capped so the two pinned output slots stay bounded: the slot is `chunk × (4 + cap × sizeof(Cluster))`, which at 9×9 and cap 1 700 is 544.5 kB per frame. `chunk_size_for(n)` applies both rules; pass the result to `reserve_output_slots()` to pre-pay the page-locking outside any timed region.

The saving is $\min(\text{GPU}, \text{host})$ per chunk, so `opt5` pays most when the two terms are comparable, $\times 1.31$ at 3×3, and least when one dominates, $\times 1.20$ at 9×9, where the host term is roughly twice the GPU term. That gap is the diagnosis that motivates `opt6` (report §8.2).

The fault model, tested against every step

• Slide 26 fits **0.68 µs per first-touch fault** on the **3×3 f32** ladder, where wall = steady-state + faults × 0.68 µs reproduced a 6.110 s run to **1 ms**. Every row below is **9×9**, and an **f64-vs-f32 gap**: a different cluster size and a different comparison, so the rate is applied **out of sample**, never refitted.

STEP	F64 WARM (FAULTS)	F32 WARM (FAULTS)	Δ WALL	Δ FAULTS	PREDICTED	VERDICT
opt3	82.44 µs (128 k)	95.66 µs (521 k)	+13.22	+393 k	+13.37	the allocator
opt4	79.83 µs (128 k)	75.20 µs (127 k)	−4.63	−0.5 k	−0.02	clean
opt5	66.39 µs (152 k)	61.85 µs (10 k)	−4.54	−141 k	−4.81	not separable
opt6	30.01 µs (0)	25.14 µs (0)	−4.87	0	0.00	clean

opt3 is the whole argument in one row. A 393 k fault gap predicts +13.37 µs; +13.22 was observed, agreement to **1 %**. The −40 % kernel is in there somewhere, invisible under 13 µs of the OS zeroing pages. Only opt4 and opt6, where the fault term is ~0, report the typedef at all.

the 9×9 ladder at cap 1 700, warm = best of reps 1-4, faults are that rep's own getrusage minor-fault count. Predicted = Δfaults × 0.68 µs ÷ 20 000 frames, with 0.68 carried in unchanged from the 3×3 fit (slide 28): nothing on this slide is tuned to make the columns agree. Observed vs predicted: +13.22 / +13.37, −4.63 / −0.02, −4.54 / −4.81, −4.87 / 0.00. This table replaces an earlier figure that quoted opt3's +16 % as a measurement of the result path; it is a measurement of two allocator states.

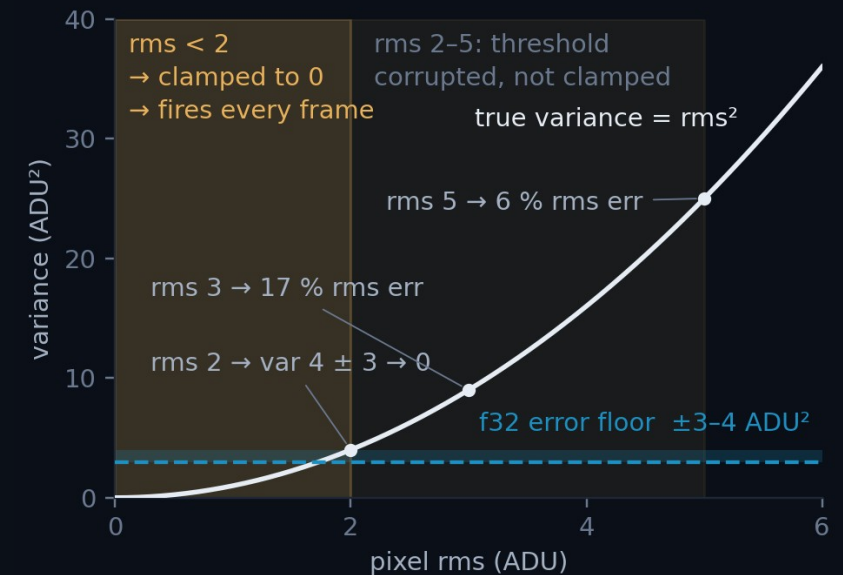
The rewrite in full, and which pixels the error reached

- Freeze a per-pixel baseline $X_0 = \text{round}(\text{mean})$ once, at the end of pedestal training, and never move it again.
- Accumulate the **centred** value $Y = X - X_0$ instead of X , and report the mean as $X_0 + \text{sum}/n$, so nothing downstream changes.
- Both operands of the subtraction are now **O(rms)-sized**: the huge common term is gone before the subtraction rather than after it.

clusterfinder_kernel.cuh

```
// before: both terms ~2.17e7, answer ~2000
var = sum2/n - mean*mean;

// after: centred on a frozen per-pixel offset X0
DEVICE_PED_TYPE resid = mean - d_pd_off[i];    // ~0(1)
DEVICE_PED_TYPE var_px = sum2[i]/n - resid*resid; // no cancellation
```



Result: the 100 % FP32 build matches the FP64 build to 3×10^{-7} , 70 clusters out of 233 million.

Which pixels the $\pm 3 \text{ ADU}^2$ floor actually reaches. Below $\text{rms} \approx 2$ the variance is lost outright; from 2 to 5 the threshold is corrupted but not clamped. The naive f32 build's +28.06 % excess is this shaded band, integrated over the sensor.

X_0 must never be updated. The accumulators are defined relative to it, so moving it invalidates every sum already collected. Welford's online variance is the other correct answer; this one was chosen because it is two lines and does not change the update's arithmetic cost.

Three ways a GPU benchmark lies

First-touch page faults

Each run materialises ~10 GB of clusters. The first pass faults in ~2.6 M pages at 0.7 μ s each, up to 4 s of pure OS work inside the timer.

Fix: re-run until getrusage() minor faults plateau (< 200 k).

CUDA-event kernel timing

avg_kernel_time_ms() measures elapsed time on a stream, including waiting for other streams. Under 8-stream load it over-reads by up to 3.5 $\times$.

Fix: Nsight Systems per-instance times; 1 stream for exclusive numbers.

The profiler itself

Under nsys, wall time per frame inflates ~4 $\times$ from API tracing.

Fix: GPU op times from nsys, wall times from unprofiled runs.

THE FAULT PROTOCOL · BRACKET EVERY TIMED CELL

```
# bracket every timed cell:
mf0 = resource.getrusage(resource.RUSAGE_SELF).ru_minflt
... t = time.perf_counter() - t0 ...
print(f'minor faults: {mf1-mf0:,}') # quote the run where this plateaus
```

Validated: **wall = steady-state + faults $\times$ 0.68 μ s** reproduced a 6.110 s run to within **1 ms**. Kernel time stayed constant throughout; the GPU was never the variable.

First-touch page faults: two sources, one counter

- A page exists in the process's address space but has no physical frame yet. On first touch the kernel finds one, **zeroes it** (mandatory), and maps it. No disk I/O: ru_majflt stays 0 all campaign. At 4 kB/page, **1 GB = 262 144 faults**.

	(A) RESULT HEAP	(B) PINNED D2H SLOTS
allocator	malloc → mmap, one ClusterVector per frame	cudaMallocHost, in submit_batch
cost / page	0.7 μs	1.0 μs: same fault + pin + DMA map
recurs?	yes, every alloc/free cycle above the mmap threshold	no: once per buffer, for its lifetime
removed by	collect_view(): it allocates nothing	nothing; reserve_output_slots() only moves it out of the timer

The two are exactly additive. Reserving subtracts precisely the pre-pin count from run 0 and changes nothing else.

NOT A CORRELATION, AN IDENTITY

3x3: 572 292 - 455 129 = 117 163 vs 117 192 pre-pin
9x9: 2 759 037 - 2 278 567 = 480 470 vs 480 474
closed form: 2 slots x 2000 x 120 004 B / 4 kB = 117 191

At 9×9 the heap never plateaus. ~9.3 GB per pass is above glibc's mmap threshold, so it is munmap'd and re-faulted every pass: ~292 k faults ≈ **10 μs/frame, permanently**. No number of re-runs removes it, which is an independent argument for opt6.

CUDA events measure the stream, not the kernel

- `avg_kernel_time_ms()` brackets the launch with **cudaEventRecord on the kernel's own stream**. What it returns is elapsed time on **that stream's timeline**, which includes time spent **queued behind other streams**.
- So it is honest at 1 stream and inflates under saturation: up to **3.5x** at 8 streams. The tell is that the derived `*PCle + overhead* = wall/N - kernel_ms` **goes negative**: kernels overlap, so `wall/frame < kernel/frame`.
- It is also not free: **~3.6 µs/frame**, i.e. 10-15 % of end-to-end throughput at 3x3, to produce a number that is unusable exactly when it matters.

ClusterFinderCUDA.hpp

```
if (m_time_kernels)                // OFF by default, all 3 finders
    cudaEventRecord(start[slot][i], sc.stream);
find_clusters_in_single_frame<<<grid, block, shmem, sc.stream>>>(...);
if (m_time_kernels)
    cudaEventRecord(stop[slot][i], sc.stream);    // <- queue-wait lands here
```

The flag exists for **comparability**, not preference: with events on for one finder and off for another, the step between them absorbs the tax.

9x9 KERNEL · F64 · NSYS

S1 · DURATION

39.86 µs

S4 · OCCUPANCY

32.66 µs

OVERLAP FACTOR

1.32x

Same kernel. The s4 column is the union of kernel intervals per frame, not how long one kernel takes. Quote s1 for duration; s4 feeds the floor, subject to the sustained-rate rule on slide 21. Full grid: A1.

Where nsys is sound, and where it is not

- Tracing a **host** call means running a callback on entry and on exit, **inside the interval being measured**. GPU work is different: the hardware stamps its own start/end and the host reads those records **afterwards**, so nothing is injected into the execution path.

MEASUREMENT	SQLITE TABLE	9×9 [F64 · S1 · CAP 1500]	VERDICT
cudaLaunchKernel: the host call	CUPTI_..._RUNTIME	1.85 μs	inflated ~4×
the kernel executing	CUPTI_..._KERNEL	39.93 μs	sound to ~2 %
cudaMemcpyAsync: the host call	CUPTI_..._RUNTIME	1.65 μs	inflated ~4×
the H2D / D2H transfer	CUPTI_..._MEMCPY	13.25 / 19.44 μs	sound to ~2 %

Same cudaMemcpyAsync, two numbers: 1.65 μs to ask for the copy, 13.25 μs for the copy to happen. If the profiler must be present at the moment to measure it, it distorts it; if the hardware records it anyway and the profiler reads it later, it does not.

Every headline number in this deck (kernel times, transfer times, duty cycles, overlap and every engine floor) comes from the _KERNEL and _MEMCPY tables (see GPU-side timestamps). Proof that side is sound: opt7 sustains 25.14 μs unprofiled against a 25.24 μs estimate measured under the profiler, 0.4 % apart, which a 4× distortion could not survive. Δ Cap 1500: the only such numbers left in the deck. Why, and what the shipped bar is, in the notes.

There is a third test

- Slide 4 showed the decision with **two** tests, which is the right simplification for the arc. The finder has **three**, and the missing one is what the next slide is about.

ClusterFinder.hpp:104-142 · all three branches

```
v = frame[i] - ped_mean[i];  rms = ped_rms[i]
if (v < -nSigma*rms)        -> skip, no update
m    = max over the 3x3 window
total = sum over the 3x3 window
if (m > nSigma*rms)          // TEST 1
    if (v == m) -> emit cluster // local-max gate
    else       -> shadow, no update
else if (total > c3*nSigma*rms) // TEST 3
    if (v == m) -> emit cluster
    else       -> update pedestal
```

WHAT EACH TEST ASKS

TEST 1 · NSIGMA · RMS

is any ONE pixel bright?

TEST 3 · C3 · NSIGMA · RMS

is the WINDOW bright?

LOCAL-MAX GATE

is this pixel the peak?

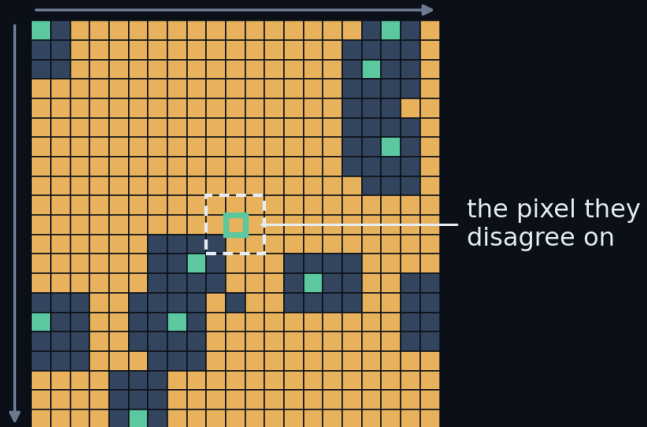
Test 1 reads the MAX of the window; Test 3 reads its SUM. That one difference is the whole of the next slide.

$c3 = \sqrt{(3 \times 3)} = 3$ is not a fudge: it falls out of variance addition. For independent samples the variance of a sum is the sum of the variances, so a 3×3 window of pixels each carrying noise σ gives **$\text{Var}(\sum v) = 9\sigma^2$** , i.e. a sum whose noise is **$\sqrt{9} \cdot \sigma = 3\sigma$** . Requiring that sum to clear nSigma of ITS OWN noise is exactly **$c3 \cdot nSigma \cdot rms$** , the **same 5σ criterion as Test 1, asked of the window instead of the pixel**. So Test 3 catches a photon whose charge is shared out so widely that no single pixel reaches 5σ , but the nine together do.

Test3 makes the clusters; Test1 only moves the pedestal

- **push_fast** touches only the pixel's **own** accumulators and never reads the stencil, so the update is **order-independent**: same set of updated pixels, bit-identical pedestal. Only a differing **decision** can diverge.

the finished frame · 21 × 21 around (125, 245)
raster order



- stored: this pixel IS the window max, and it clears 5σ
- shadow: the window max clears 5σ , but this pixel is not it
- pedestal sample: nothing in the window clears 5σ
- the disputed pixel — serial sampled here, frozen stored

the moment (125, 245) was tested · only the 4 already-scanned neighbours differ

31.669	-14.420	33.763
31.637	-14.405	33.730
41.680	56.473	30.007
15.214	40.217	21.979

frozen above serial · amber ring = pushed pedestal

serial $\Sigma = 256.489 < 256.511 \rightarrow$ pedestal sample

frozen $\Sigma = 256.581 > 256.511 \rightarrow$ CLUSTER

max = 56.473 in both $\rightarrow$ Test1 never moved

Measured two ways. **Instrumented**: of the 19 disagreeing clusters, the **11 that only frozen finds are all Test 3**; the 8 only serial finds are all the local-max gate. **Ablated**: compile Test 3 out and the 11 go to **zero**.